Chapter 5- Structured Design

5.1. Introduction to Structured design

5.2. Physical Architecture Layer design

5.3. Client-Server System/Software Design

5.4. Interface design

5.5. Software Reuse/Component/ design

5.6. Data Structure /Data Storage Design

5.7. Algorithm/Program design

Chapter 5- Structured Design

5.1. Introduction to structured design …

Design phase:

- Determines how system will be implemented & provides detailed plans for all system components;
- Uses requirements as input to create a blueprint for the future system.
  - Project team converts business requirements into **system requirements** (that describe the technical details for building the system).
- System requirements: are communicated through a collection of design documents and *physical process & data models*.
  - design docs. & physical models make up blueprint for new system.
- ***Business analysts :*** turn their attention to redesigning of work flows & procedures that users will follow when using the new system, while *systems analysts* focus on more technical design elements.
- Ultimately, redesigned business processes & procedures will be communicated in user documentation & training materials (when delivering the system).

Chapter 5- Structured Design

Activities of the Design Phase

Activities in the Design Phase	Deliverables
Determine preferred system acquisition strategy: make, buy or outsource	Alternative matrix
Design the architecture of the system	Architecture Design
Make HW and SW Selections	HW & SW specifications
Design System Navigation, inputs & outputs	Interface Design
Convert Logical process models to physical models	Physical process model
Design programs that will perform the system processes	Program design specifications
Convert logical data models to physical data model	Physical Data Model
Revise CRUD matrix	CRUD matrix
Design the way in which data will be stored	Data Storage Design
Compile final system specification	System Specification: all of the above deliverables combined and presented to approval committee

Chapter 5- Structured Design

5.1. Intro to structured design: System Acquisition Strategies

- Assumption up-to the analysis phase: The system will be designed, developed, and implemented by the project team.
- In many projects, however, team may recognize that some parts or even all of the new system's software will be acquired from an outside provider.
- It is essential to know what we need (requirements) before seeking a product that provides the best fit.
- Otherwise, we run the risk of letting a SW vendor tell us what we need & we obtain SW that does not fulfill our real business & user requirements.
- There are, however, actually three primary ways to approach the creation of a new system: (1) develop a custom application in-house; (2) buy a packaged system and (possibly) customize it; and (3) **Outsource**: rely on an external vendor/developer/service provider to build/provide the system. Each of these choices has its strengths and weaknesses (see next slide)

Chapter 5- Structured Design

5.1. Intro to structured design: System Acquisition Strategies

Dennis, Wixon & Roth, 2019, p.225

Pros	Cons
Custom Development	
Get **exactly** what we want New system built consistently with existing technology and standards Build and retain technical skills and functional knowledge in-house Allows team flexibility and creativity Unique solutions created for strategic advantage	Requires significant time and effort May add to existing backlogs May require skills we do not have Often costs more Often takes more calendar time Risk of project failure
Packages (purchased or obtained from ASP or SaaS)	
No need to "reinvent the wheel" for common business needs Tested, proven product Cost savings Time savings Utilize vendors' expertise Some customization may be possible	Rarely a perfect fit Organizational processes must adapt to software Reliance on vendor for maintenance and future enhancements Will not develop in-house functional and technical skills Unique needs may go unmet May require system integration
Outsourced Development	
Hire expertise we do not have May save time and money Lower risk	No opportunity to build in-house expertise Reliance on vendor Future options limited Security—potential loss of confidential info Performance based on contract terms

Summary of software acquisition options pros and cons.

Chapter 5- Structured Design

5.1. Intro to structured design: Avoiding Classic Design Mistakes

Four design phase classic mistakes & how to avoid them

1. **Reducing design time:** If time is short, there is the urge to exclude "unproductive" design activities so that team jumps into "productive" programming. This leads to missing important details that have to be investigated later at a much higher time cost (usually, at least 10 times longer). **Solution:** If time is short, use RAD/Agile techniques & time-boxing to eliminate functionality or move it into future versions.
2. **Feature creep:** Even if you are successful at avoiding scope creep, about 25% of requirements will still change. Changes—big & small—can significantly increase time & cost. **Solution:** Ensure that all changes are vital & that the users are aware of the impact on cost & time. Try to move proposed changes into future versions.
3. **Silver bullet syndrome:** marketing claims that some design tools solve all problems & magically reduce time & costs. No one tool can eliminate overall time or costs by more than 25% (some can reduce individual steps by this much). **Solution:** If a design tool has claims that appear too good to be true, just say no.
4. **Switching tools in mid-project:** analysts may switch to what appears to be a better tool during design in the hopes of saving time or costs. Usually, any benefits are outweighed by the need to learn the new tool (including even "minor" upgrades to current tools). **Solution:** Do not switch/upgrade unless there is a compelling need for specific features in the new tool, and then explicitly increase the schedule to include learning time.

Chapter 5- Structured Design

5.2. Physical Architecture Layer Design (PALD)

- Most Info Systems are spread across two or more computers. Example:-

  1. Web-based system: run in a user's browser on a PC, but interacts with Web server & possibly other computers over the Internet.

  2. An in-house system runs on a company's network with a program installed on your PC, but interact with a DB server elsewhere on the company network.

- An important step of the design phase is the creation of the **architecture design (AD)**: plan for how IS components will be distributed across multiple computers & what HW & SW (OS: e.g., Windows or Linux; DB-e.g., Oracle, MS-SQL Server or MySQL, etc) will be used on each computer
- Thus, AD **describes a system's HW, SW, & network environment**.
- Flows primarily **from nonfunctional requirements**
- AD **deliverables**: **the architecture design & the HW & SW specification**.
- Performed by experienced System Architects (may be outside consultants)

Chapter 5- Structured Design

5.2.1. Elements of a Physical Architecture Layer Design (PALD)

- **Objective of PALD** : is to **determine how the SW components of the IS will be assigned to the HW devices of the system**.
- It is important to know major functions of the SW (to understand how the SW can be divided into d/t parts) & major types of HW onto which the SW can be placed.
- Although there are many ways in which the SW components can be placed on the HW components (like server-based & client-based architectures), **most common architecture is the client–server architecture (discussed later)**.

5.2.1.1. Architectural Components

- **Major architectural components of any system** are the **SW & the HW**.
- Team need to Identify Major SW components of the system being developed & then allocate to various HW components (the system will operate on).
- 4 basic **layered functions of SW** systems:
  - ***Data storage*** (data entities on **ERDs**),
  - ***Data Access Logic*** (e.g., Structured Query **Language-SQL**),
  - ***Application Logic*** (**documented on DFDs**, requirements list) &
  - ***Presentation logic*** (display of info to user & accept user's commands, i.e., **UI**).

Chapter 5- Structured Design

5.2.1. Elements of a Physical Architecture Layer Design (PALD)

- These 4 functions are the basic building blocks of any information system
- **3 primary HW components** of a system are: **Clients, Servers & Network**
  - ✓ **Client computers:** I/O devices- desktop, laptop, handheld /tablet devices, smart phones, special purpose terminals,…
  - ✓ **Servers** : multi-user computers used to store SW & data, accessed via permission
  - ✓ **Network:** connects Clients & Servers. Network can vary in speed from **slow cell phones/modem** connections that must be dialed, to medium-speed always-on **frame relay networks**, to **fast always-on broadband** connections

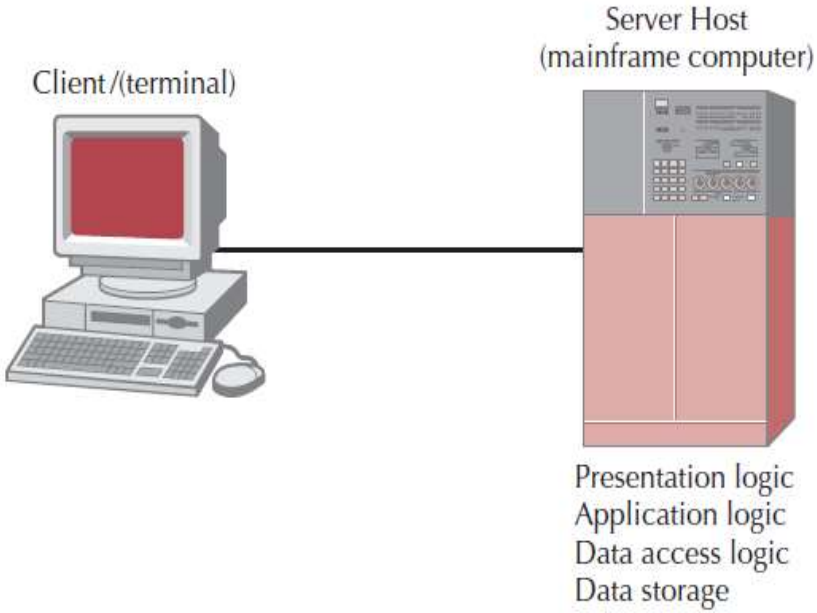
A. Server-Based Architecture

- 1st computing architectures (usually a **central mainframe computer**) performing all four functions. Clients (usually terminals) enabled users to send & receive messages to & from the server computer.
- Clients merely captured keystrokes & **sent them to the server for processing** and accepted instructions from the server on what to display.

Chapter 5- Structured Design

A. Server-Based Architecture…

- ✓ Simple architecture & works well.
- ✓ Server: hosts All components
- ✓ Problem: server must process all messages.
- ✓ As demands for more applications grow, many server computers become overloaded & unable to quickly process all the users' demands.
- ➢ As Response time becomes slower, network managers are required to spend increasingly more money to upgrade the server computer.
- ✓ Unfortunately, **upgrades come in large increments and are expensive**; it is **difficult to upgrade "a little."**



Dennis, Wixon & Tegarden, 2012, p.475

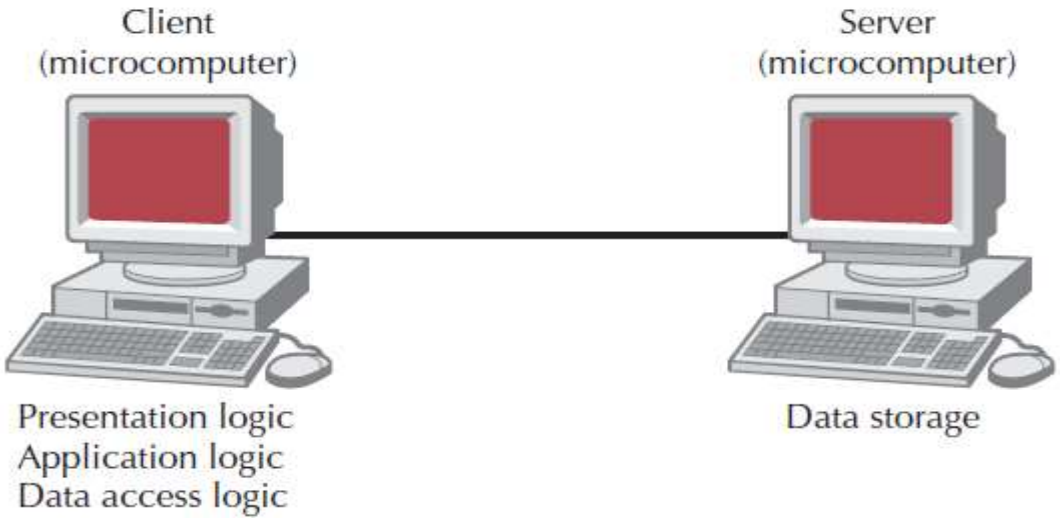
Chapter 5- Structured Design

B. Client-Based Architecture

- Clients are PCs on a LAN, & server computer is a server on same network.
- Client: responsible for the presentation, application & data access logics
- Server: simply stores the data
- This simple architecture also often works well.
- However, **as more network applications grow, network becomes overloaded.**
- **Problem**: all data on the server must travel to the client for processing.

Example: suppose user wishes to display a list of all employees with company life insurance. All the data in the database must travel from the server where the database is stored over network to the client, which then examines each record to see if it matches the data requested by the user. This **can overload both the network and the power of the client computers.**

Dennis, Wixon & Tegarden, 2012, p.476



Chapter 5- Structured Design

C. Client–Server Architectures

- Most organizations today are moving to client–server architectures
- **Attempt to balance processing b/n client & one or more server devices**.
- *Client:* is responsible for **presentation logic**.
- *Server:* is meant for **data access logic & data storage**.
- **Application logic**: may **reside on either client or server** or be split b/n both
- Thick/fat client: contains **all or most of application logic**,
- **Thin clients:** contains a **small portion of application logic**, are popular because of lower overhead and easier maintenance.

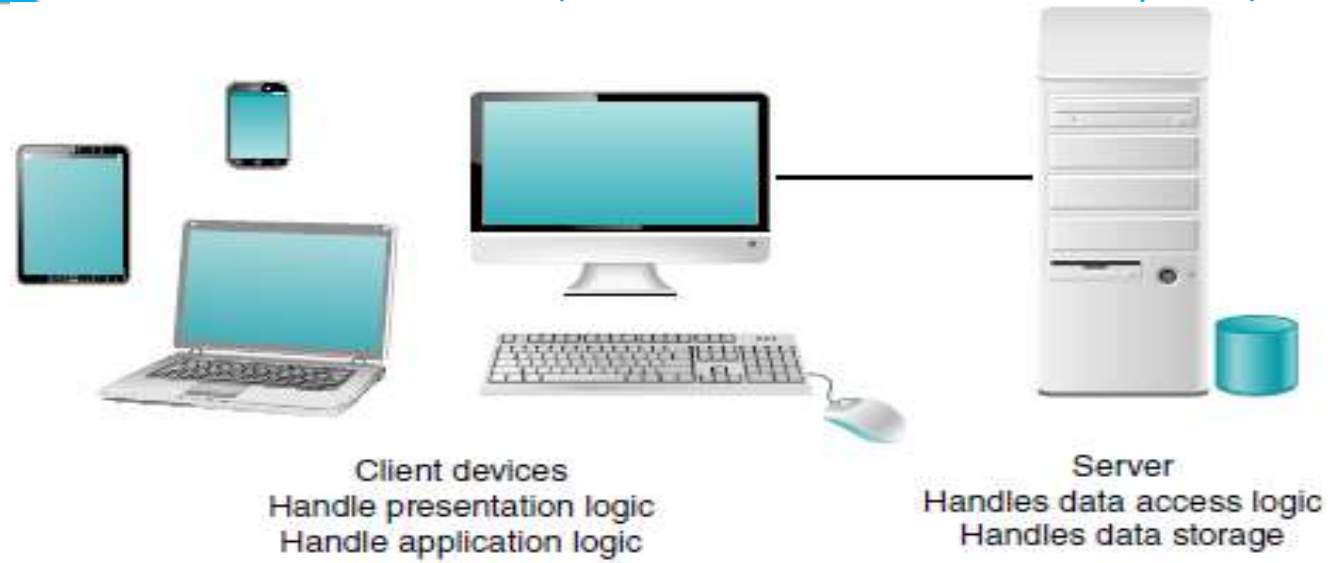
Example:

- In many Web-based systems:
  - ✓ the **Web browser performs presentation and only minimal application logic using say JavaScript,**
  - ✓ the **Server side** has most of the **application logic, all of the data access logic, and all of the data storage.**

Chapter 5- Structured Design

C. Client–Server Architectures…

Two-tiered Client-Server Architecture (Dennis, Wixon & Roth, 2019, p.242)



Client–server architectures have four important benefits:

1. **Scalability**: easy to increase/ or decrease the storage & processing capabilities of the servers. If one server becomes overloaded, simply add another server so that many servers are used to perform application logic, data access logic, or data storage.

✓ Cost to upgrade is gradual, & you **can upgrade in small increments** rather than spending **hundreds of thousands to upgrade a mainframe server**.

Chapter 5- Structured Design

C. Client–Server Architectures…

2. Support different types of clients & servers: It is possible to **connect computers that use different operating systems** so that you are **not locked into one vendor.** Users can choose the type of computer they prefer (e.g., **combining both Windows & Apple Macintosh computers on same network**).

Middleware: system SW designed to translate between different vendors' SW.

- **Installed on both client & server computers**.
- Client SW communicates with middleware, which can **reformat the message** into a **standard language** that can be understood by the middleware, which assists the server software. **Analogy: human translator knows source-target langs**.

3. Simplicity of clearly **separating presentation, application & data access logics** (for thin client–server architectures that use Internet standards) and design each to be somewhat independent.

E.g., Presentation logic can be **designed in HTML/XML to specify page appearance** (e.g., **colors, fonts, order of items, specific words used, command buttons**,…; see interface design later).

- Simple **program statements** are used to link parts of interface to specific application logic modules that perform various functions.

Chapter 5- Structured Design

C. Client–Server Architectures…

These **HTML or XML files defining the** interface can be changed without affecting the application logic. Likewise, it is **possible to change the application logic without changing the presentation logic or the data**, which are stored in databases & accessed by SQL commands.

4. **Less impact of failure:** i.e., if a server fails, **only applications requiring that server will fail**. **The failed server can be swapped out and replaced and the applications can then be restored**.

Critical limitation of Client–server architectures: **Complexity**

- All applications in client–server computing have **two parts—the SW on client and the SW on the server side.** Writing this SW is more complicated **than writing the traditional all-in-one SW used in server-based architectures** (discussed earlier).
- **Updating** the overall system with a new version of the software **is more complicated, too**.; need to update all clients and all servers and **you must ensure that the updates are applied on all devices.**

Chapter 5- Structured Design

C. Client–Server Architectures… Client–Server Tiers

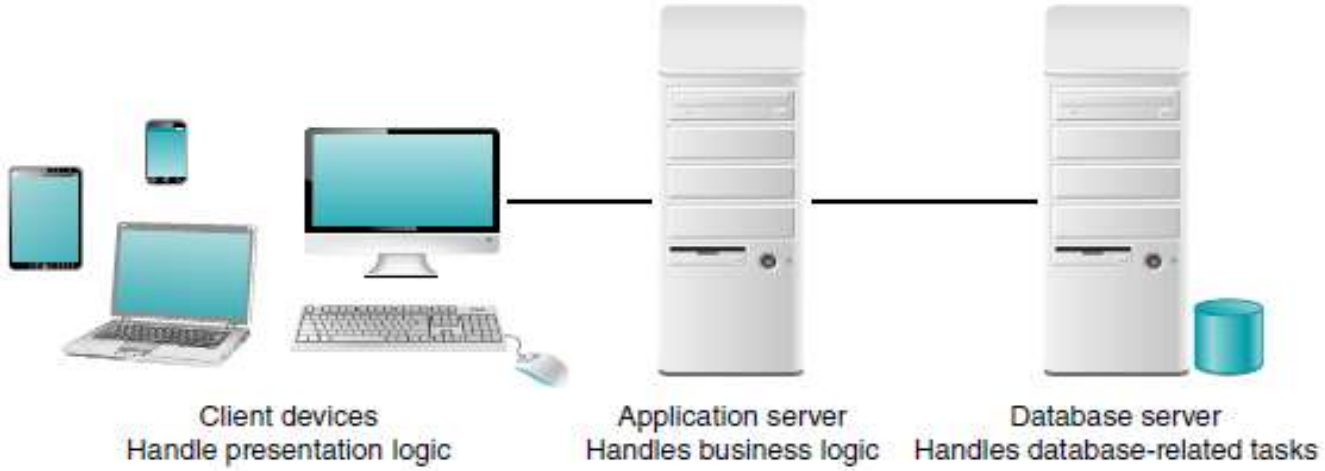
There are many ways of **partitioning application logic between the client & the server.**

1. **2-tiered architecture- uses only two sets of computers**: **Clients & Servers**.

- Most common configuration in which server handles data and client handles application & presentation (shown on a previous slide).

2. **3-tiered architecture uses 3 sets of computers**: the SW on the **client** is responsible for **presentation logic**, an **application server(s)** is responsible **for the application logic**, & a separate **database server(s)** is responsible **for the data access logic and data storage**. The **middle tier may be divided into tiers** itself, resulting in an overall architecture called an "**n-tier architecture.**"

Three-tiered Client Server Architecture (Dennis, Wixon & Roth, 2019, p.243)

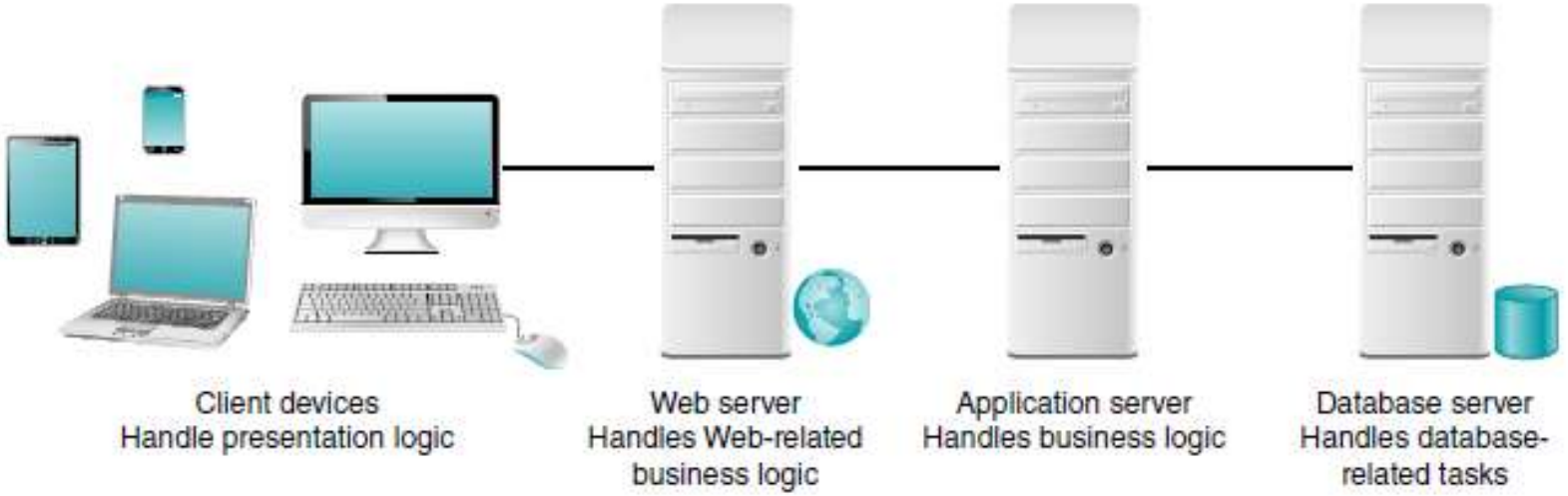


Chapter 5- Structured Design

C. Client–Server Architectures… Client–Server Tiers

3. **n-tiered architecture:** distributes the work of the application (the middle tier) among multiple layers of more specialized server computers.

N-tiered Client-Server Architecture (Dennis, Wixon & Roth, 2019, p.244)



Common in today's Web-based e-commerce systems.

1st , The browser software on client computers makes HTTP requests to view pages from the Web server(s), and;

2nd , The Web server(s) enable the user to **view merchandise for sale** by responding with HTML documents.

Chapter 5- Structured Design

C. Client–Server Architectures… Client–Server Tiers

3rd, As **user shops**, components on app. server(s) are called as needed to allow user to put items in a shopping cart; **determine item pricing & availability**; compute purchase costs, sales tax, & shipping costs; **authorize payments**, etc. **Such business logic, or detailed processing, are stored on application server(s)** & are accessible to any application.

4th, The **database server(s)** ***manage the data components*** of the system.

- The four components are separate: **making it easy to spread d/t components on d/t servers & to partition the application logic on a Web-oriented server and a business-oriented server.**

Advantage of an n-tiered client–server architecture over a 2/3-tiered:

- ❑ It **separates out processing** with better **balancing of the load** on different servers; it is more scalable.
- ✓ If application server is too heavily loaded, simply replace it with a more powerful server or just put in more application servers to share the load.
- ✓ Conversely, **if database server is underused, we could store data from another application on it**.

Chapter 5- Structured Design

C. Client–Server Architectures… Client–Server Tiers

There are ***two primary disadvantages to an n-tiered architecture*** **compared with a two/three-tiered architecture.**

1st , the **configuration puts a greater load on the network**. Thus, the n-tiered model **requires more communication among the servers**; it **generates more network traffic;** **=>** needs a higher capacity network.

2nd, it is **much more difficult to program & test software** **because more devices have to communicate properly to complete a user's transaction**.

D. Mobile Application Architectures

- System architect must **decide how much of the presentation, business, & data access logics** will reside on **mobile device & on server device**.
- If **application requires local processing using mobile device's resources (e.g., camera or GPS)**, & is only occasionally connected to the server, it should be **created as a rich client.**
  - With **rich clients**: **business + data access + presentation logics** are included on the device.

Chapter 5- Structured Design

D. Mobile Application Architectures…

Mobile applications Dev't: Unlike desktop/laptop clients having fairly uniform capabilities, mobile devices offer a **wide array of screen sizes, presentation formats, & device capabilities**.

Mobile device market is fragmented into:

- ✓ **tablet & smart-phone segments** with an array of options in each segment.
- ✓ **Devices are rapidly evolving** & **dev't env't choices are also very dynamic**.
- ❑ Development Technology choices for **mobile applications**:
  - **Native applications,**
  - **Cross-platform frameworks, &**
  - **Mobile Web apps.**

I. **Native applications**: **written** to **run on a specific device with a specific OS.**

- ✓ Applications are **written in a device's native code**:

  E.g., **Objective C for Apple devices**; **Java for Android devices**

Advantage: * **Provide richest user experience** &

* **Can take full advantage of the device's resources.**

Chapter 5- Structured Design

D. Mobile Application Architectures…

Disadvantages/Limitations:

* Native applications must be re-built for each native OS
* Developers with skills in multiple languages are scarce.
* **Apps must be updated as new device models** & OS versions are released.

II. **Cross-platform frameworks** : enable developers to **create a mobile app in Web-based techs like Java-script/HTML & use the** framework to adapt the app for multiple devices. Usually app must be "**tweaked**" for each device on which it will be deployed.

- Mobile apps developed with this approach **will not provide the same rich user experience as the native apps**, so it is **best to use this approach with informational applications that do not require heavy use of device functions**.

III. **Mobile Web apps**: are **platform independent** & can be **deployed on any device equipped with the Web browser.**

- **Mobile Web apps** are built with Web technology **such as HTML**. These apps:
  * Cannot use the hardware & software on the device,
  * Require an **Internet connection to work**, and
  * Will provide the most **generic user experience**.

Chapter 5- Structured Design

Advances in Architecture Configurations

- Advances in HW, SW & networking have given rise to a # of **new architecture options**. Two important advances: **virtualization & cloud computing**

A. Virtualization: creation of a virtual device such as a **server or storage device**. Good Analogy: **partitioning 1 physical hard drive-to treat each partitioned, "virtual" drive, as if it is a distinct physical hard drive**.

- This concept has become common & we hear about:
  - **Server** virtualization,
  - Storage virtualization,
  - Network virtualization…

A1. Server virtualization: partitioning a physical server into smaller virtual servers.

- **SW** is used to divide physical server into multiple virtual env'ts, called virtual/private servers
- **overcomes the primary limitation of older style server-based architectures that were based on single, large, expensive, monolithic computers;**
- Virtual servers are independent of each other, but co-reside on same physical server.

Chapter 5- Structured Design

Advances in Architecture Configurations …

- Each **virtual server runs an operating system & can be rebooted independently of the other virtual servers**.

Advantage: **Less HW is required** to provide a set of virtual servers as compared to equivalent physical servers, so costs are reduced. It also optimizes the utilization of the physical server, saving on operational costs.

Gartner survey of **mid-sized businesses found** that over 75% of midsized businesses planned to make widespread use of server virtualization by 2012. => Aiming to lower costs, improve utilization, & increase availability.

A2. Storage virtualization: involves ***combining multiple network storage devices into what appears to be a single storage unit***.

A **storage area network (SAN):** uses storage virtualization to create a high-speed sub-network of shared storage devices.

- tasks such as **backup**, **archiving**, and **recovery are easier and faster.**

A3. Network Virtualization/NV/: refers to abstracting network resources that were traditionally delivered in hardware to software. NV **can combine multiple physical networks to one virtual, software-based network, or it can divide one physical network into separate, independent virtual networks**.

Chapter 5- Structured Design

Advances in Architecture Configurations …

- NV software allows network administrators to move virtual machines across different domains without reconfiguring the network.

Example:- A **VLAN** is a subsection of a local area network (LAN) created with software that **combines network devices into one group, regardless of physical location**.

- VLANs can **improve the speed and performance of busy networks** and **simplify changes or additions to the network**.

B. Cloud Computing (CC): It is no longer necessary for organizations to own, manage, & administer their own computing infrastructure.

- **CC** is an **arrangement wherein everything, from computing power/ infrastructure, applications, business processes to personal collaboration** can be delivered as a service wherever & whenever needed.
- The "cloud" in CC can be defined as *set of HW, networks, storage, services, & interfaces that combine to deliver aspects of computing as a service*.

Chapter 5- Structured Design

Advances in Architecture Configurations …

- Cloud services: include the **delivery of SW, infrastructure, & storage *over the Internet*** (as separate components/a complete platform) based on user demand.
- CC can be implemented in 3 ways: private, public, or hybrid clouds.

Public clouds, services are **provided "as a service" over the Internet with *little or no control over the underlying technology infrastructure***.

*Private clouds offer activities & functions "as a service," but are **deployed over a company intranet or hosted data center***.

Hybrid clouds combine the power of both public & private clouds; **activities and tasks are allocated to private or public clouds as required**.

Advantages of CC:

- ***Scalability/elasticity***: resources can be increased/decreased based on demand. The cloud can **scale up for periods of peak demand & scale down for times of less demand**. Applications in the cloud can scale up as users are added & when the application's requirements change.

Chapter 5- Structured Design

Advances in Architecture Configurations …

- ***Flexibility***: Arrangements are made with cloud service provider for a certain amount of computing, storage, software, process, or other resources. After using these resources, they can be released if no longer required.
- ***Standardized interface***: cloud services typically have standardized **application program interfaces (APIs).** This capability lets the customer more easily create linkages between cloud services.
- ***Cost reduction***: Usage of the cloud is measured & customers pay only for resources used—much like your use of electricity in your home; making CC very attractive from a financial perspective.
- **CC suppliers utilize virtualization as a key enabling tech**. Through the CC model, the power of virtualization is converted into measurable bus. value.

Challenges/disadvantages of CC:

- Although the benefits of the CC model are many (scalability, cost reduction, device independence, performance, and more), it is still in its *infancy* and companies are still learning how best to utilize it.

Chapter 5- Structured Design

Advances in Architecture Configurations …

E.g., Recently, Amazon (a major CC supplier), experienced a serious failure that affected several organs that depended on its's CC to run their businesses.

- ❑ Therefore, organizations should be prepared to carefully structure their CC arrangements & include redundancy in their apps so that the negative consequences of a catastrophic failure are minimized.

Comparing Architecture Options

- ❖ No architecture is inherently better than the others.
- ➢ It is important to understand strengths & weaknesses of each computing architecture and when to use each (Dennis, Wixon & Tegarden, 2012, p.480)

Characteristic	Server-based	Client-based	Client-Server
Cost of Infrastructure	Very high	Medium	Low
Cost of app development	Medium	Low	High
Ease of development tools	Low	High	Low to medium
Interface capabilities	Low	High	High
Control & security	High	Low	Medium
Scalability	Low	Medium	High

Chapter 5- Structured Design

5.2.2. Creating an Architecture Design(CAD)

- CAD begins with nonfunctional requirements.
  - 1st **refine nonfunctional requirements(NFR) into more detailed requirements** & the software components to be placed on each device.
  - 2nd In a client–server architecture, one also has to **decide whether to use a two-tier, three-tier, or n-tier architecture**.
  - 3rd, **use NFRs & the architecture design to : develop the HW and SW specification**.
- 4 primary types of NFRs that can be important in designing the architecture: operational, performance, security, cultural & political requirements. We focus on points that may affect the architecture design.

1. ***Operational requirements***: specify the operating environment(s) in which the system must perform and how those may change over time.

- It usually refers to operating systems, system software etc.

Chapter 5- Structured Design

5.2.2. Creating an Architecture Design(CAD)…

Operational requirements (Dennis, Wixon & Roth, 2019, p.250)

Type of Requirement	Definition	Examples
Technical Environment Requirements	Special hardware, software, and network requirements imposed by business requirements	• The system will work over the Web environment with any browser. • All office locations will have an always-on network connection to enable real-time database updates. • A version of the system will be provided for customers connecting over the Internet via a small-screen smartphone.
System Integration Requirements	The extent to which the system will operate with other systems	• The system must be able to import and export Excel spreadsheets. • The system will read and write to the main inventory database in the inventory system.
Portability Requirements	The extent to which the system will need to operate in other environments	• The system must be able to work with different operating systems (i.e., Linux, Windows 10). • The system may need to operate with mobile devices such as an iPad.
Maintainability Requirements	Expected business changes to which the system should be able to adapt	• The system will be able to support more than one manufacturing plant upon 3 months advance notice. • New versions of the system will be released every 2 months.

Chapter 5- Structured Design

5.2.2. Creating an Architecture Design(CAD)…

2. ***Performance Requirements:*** focus on response time, capacity, & reliability

Performance Requirements **(Dennis, Wixon & Roth, 2019, p.251)**

Type of Requirement	Definition	Examples
Speed Requirements	The time within which the system must perform its functions	• Response time must be 4 seconds or less for any transaction over the network. • The inventory database must be updated in real time. • Orders will be transmitted to the factory floor every 3 minutes.
Capacity Requirements	The total and peak number of users and the volume of data expected	• There will be a maximum of 2000 simultaneous users at peak use times. • A typical transaction will require the transmission of 300 K of data. • The system will store data on approximately 50,000 customers for a total of about 2 GB of data.
Availability and Reliability Requirements	The extent to which the system will be available to the users and the permissible failure rate due to errors	• The system should be available 24/7, with the exception of scheduled maintenance. • Scheduled maintenance shall not exceed one 6-hour period each month. • The system will have 99% uptime performance.

Chapter 5- Structured Design

5.2.2. Creating an Architecture Design(CAD)…

Security Requirements: ***Security*** is ability to protect IS from disruption & data loss

Security Requirements **(Dennis, Wixon & Roth, 2019, p.254)**

Type of Requirement	Definition	Examples
System Value Estimates	Estimated business value of the system and its data	• The system is not mission critical, but a system outage is estimated to cost $150,000 per hour in lost revenue. • A complete loss of all system data is estimated to cost $20 million.
Access Control Requirements	Limitations on who can access what data	• Only department managers will be able to change inventory items within their own department. • Customer Service personnel will be able to read and create items in the customer file, but cannot change or delete items.
Encryption and Authentication Requirements	Defines what data will be encrypted where and whether authentication will be needed for user access	• Data will be encrypted from the user's computer to the website to provide secure ordering. • Users logging in from outside the office will be required to authenticate.
Virus Control Requirements	Controls the spread of viruses	• All uploaded files will be checked for viruses before being saved in the system.

Chapter 5- Structured Design

5.2.2. Creating an Architecture Design(CAD)…

4. Cultural and Political Requirements: are specific to the countries in which the system will be used. In today's global business environment, organizations are expanding their systems to reach users around the world. Although this can make great business sense, its impact on application development should not be underestimated. Cultural & Political Requirements **(Dennis, Wixon & Roth, 2019, p.254)**

Type of Requirement	Definition	Examples
Multilingual Requirements	The language in which the system will need to operate	• The system will operate in English, French, and Spanish.
Customization Requirements	Specification of what aspects of the system can be changed by local users	• Country managers will be able to define new fields in the product database in order to capture country-specific information. • Country managers will be able to change the format of the telephone-number field in the customer database.
Making Unstated Norms Explicit	Explicitly stating assumptions that differ from country to country	• All date fields will be explicitly identified as using the month-day-year format. • All weight fields will be explicitly identified as being stated in kilograms.
Legal Requirements	The laws and regulations that impose requirements on the system	• Personal information about customers cannot be transferred out of European Union countries into the United States. • It is against US federal law to divulge information on who rented what videotape, so access to a customer's rental history is permitted only to regional managers.

Chapter 5- Structured Design

5.2.3. Designing the Architecture: summaries of the relationship b/n Requirements & recommended Architectures are indicated in table:

Nonfunctional requirements and their implications for architecture design

(Dennis, Wixon & Roth, 2019, p.260)

Requirements	Server-Based	Thin Client-Server	Thick Client-Server
Operational Requirements			
System Integration Requirements	✓	✓	✓
Portability Requirements		✓	
Maintainability Requirements	✓	✓	
Performance Requirements			
Speed Requirements		✓	✓
Capacity Requirements		✓	✓
Availability/Reliability Requirements	✓	✓	✓
Security Requirements			
High System Value	✓	✓	
Access Control Requirements	✓		
Encryption/Authentication Requirements		✓	✓
Virus Control Requirements	✓		
Cultural/Political Requirements			
Multilingual Requirements		✓	
Customization Requirements		✓	
Making Unstated Norms Explicit		✓	
Legal Requirements	✓	✓	✓

Chapter 5- Structured Design

5.2.4. Hardware & Software Specification: is a document that describes what HW & SW are needed to support the application. Steps:

1. Define the software. Start with the **OS** (e.g., **Windows, Linux**) & any **special purpose SW on client and servers (e.g., Oracle DB)**. Consider **additional costs like training, maintenance, warranties, & licensing**.
2. Create list of hardware needed. Such as DB servers, network servers, peripheral devices (e.g., printers, scanners), backup devices, storage components, & other HW needed. Please note the quantity of each item needed.
3. Define minimum requirements for each piece of HW. Many organizations have standard lists of approved HW & SW that must be used. But, when team is operating in new territory & is not constrained by the need to select from an approved list, then team must convey such requirements as amount of processing capacity, storage space, & any special features to be included. Experience matters; but hints also help you describe HW needs (see next slide).

- ❑ Finally project team prepares a Request For Proposal (RFP) based on legal & policies provided by the purchasing department, which then issues the RFP.
- ❑ The project team then selects the most desirable vendor for HW & SW on basis of proposals, perhaps using a weighted alternative matrix.

Chapter 5- Structured Design

5.2.4. Hardware and Software Specification…

Sample HW & SW Specification **(Dennis, Wixon & Roth, 2019, p.261)**

	Standard Client	Standard Web Server	Standard Application Server	Standard Database Server
Operating System	• Windows 10 Pro	• Linux	• Linux	• Linux
Special Software	• Real Audio • Adobe Acrobat Reader	• Apache	• Java	• Oracle
Hardware	• 1 TB disk drive • Intel®-Core™ i5-8400 six core processor • 22-inch LED Monitor	• 8 TB disk drive • Xeon E5-4600 v4	• 8 TB disk drive • Xeon E5-4600 v4	• 32 TB disk drive • RAID • Xeon 28 core processor
Network	• Always-on Broadband, preferred	• Dual 100 Mbps Ethernet	• Dual 100 Mbps Ethernet	• Dual 100 Mbps Ethernet

Factors in HW & SW Selection **(Dennis, Wixon & Roth, 2019, p.262)**

1. **Functions and Features** What specific functions and features are needed (e.g., size of monitor, software features)?
2. **Performance** How fast do the hardware and software operate (e.g., processor, number of database writes per second)?
3. **Legacy Databases and Systems** How well do the hardware and software interact with legacy systems (e.g., can it write to this database)?
4. **Hardware and OS Strategy** What are the future migration plans (e.g., the goal is to have all of one vendor's equipment)?
5. **Cost of Ownership** What are the costs beyond purchase (e.g., incremental license costs, annual maintenance, training costs, salary costs)?
6. **Political Preferences** People are creatures of habit and are resistant to change, so changes should be minimized.
7. **Vendor Performance** Some vendors have reputations or future prospects that are different from those of a specific hardware or software system that they currently sell.

Chapter 5- Structured Design

5.3. Client-Server Systems/software Design:

- A *distributed system* is a system in which:
  - computations are performed by *separate programs*
  - … normally running on separate pieces of hardware
  - … that *co-operate* to perform the task of the system.
- *Server:*
  - A program that *provides a service* for other programs that connect to it using a communication channel
- *Client*
  - A program that accesses a server (or several servers) *to obtain services*
  - A server may be accessed by many clients simultaneously

Sequence of activities in a client-server system/software:

1. The **server starts running**
2. The **server waits for clients** to connect. (*listening*)
3. Clients start running & perform operations
   - ❖ Some operations involve requests to the server

Chapter 5- Structured Design

5.3. Client-Server Systems/software Design:...

4. When a client attempts to connect, the **server accepts the connection** (if it is willing)
5. The **server waits for messages** to arrive from connected clients
6. When a message from a client arrives, the **server takes some action** in response, then resumes waiting
7. Clients and servers continue functioning in this manner until they decide to shut down or disconnect

Alternatives to the client server software/system architecture:

- Have a *single program* on one computer that does everything
- Have *no communication*
  - Each computer performs the work separately
- Have some mechanism other than client-server communication for exchanging information

  E.g. one program writes to a database; the other reads from the database

Chapter 5- Structured Design

Advantages of Client-Server Systems

- The work can be *distributed* among different machines
- The clients can access the server's functionality from a *distance*
- The client and server can be *designed separately*
- They can both be *simpler*
- All the *data can be kept centrally* at the server
- Conversely, *data can be distributed* among many different geographically-distributed servers
- The server can be accessed *simultaneously* by many clients
- *Competing clients can be written* to communicate with the same server, and vice-versa

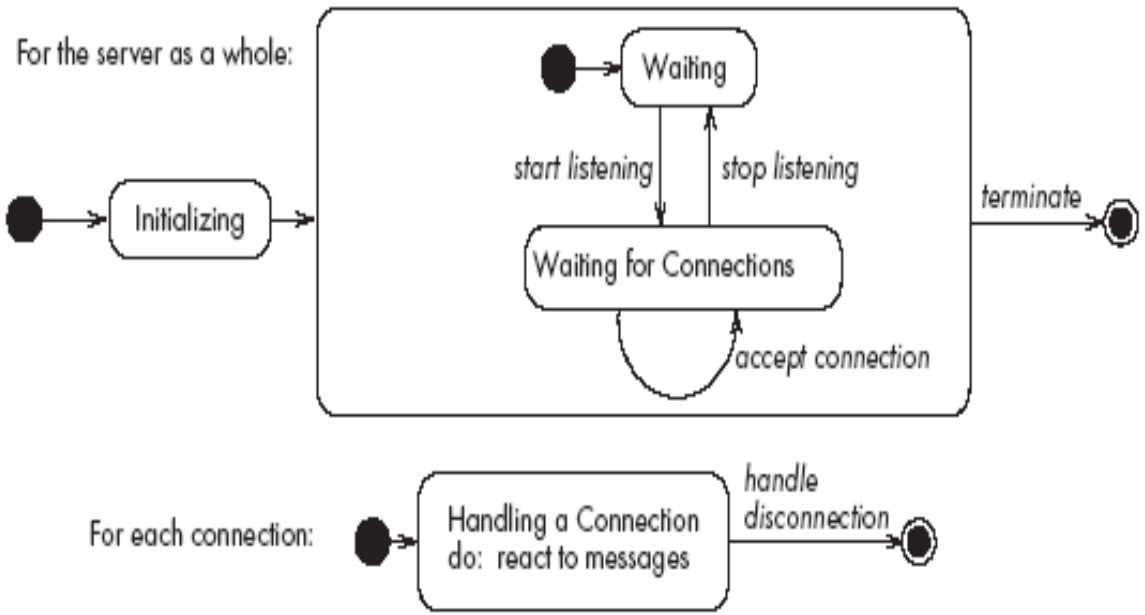
Example of client-server systems

- The World Wide Web; Email; Network File System
- Transaction Processing System (e.g., reservation, sales, etc)
- Remote Display System, Communication System, DB Systems….

Chapter 5- Structured Design

Activities of the Server:

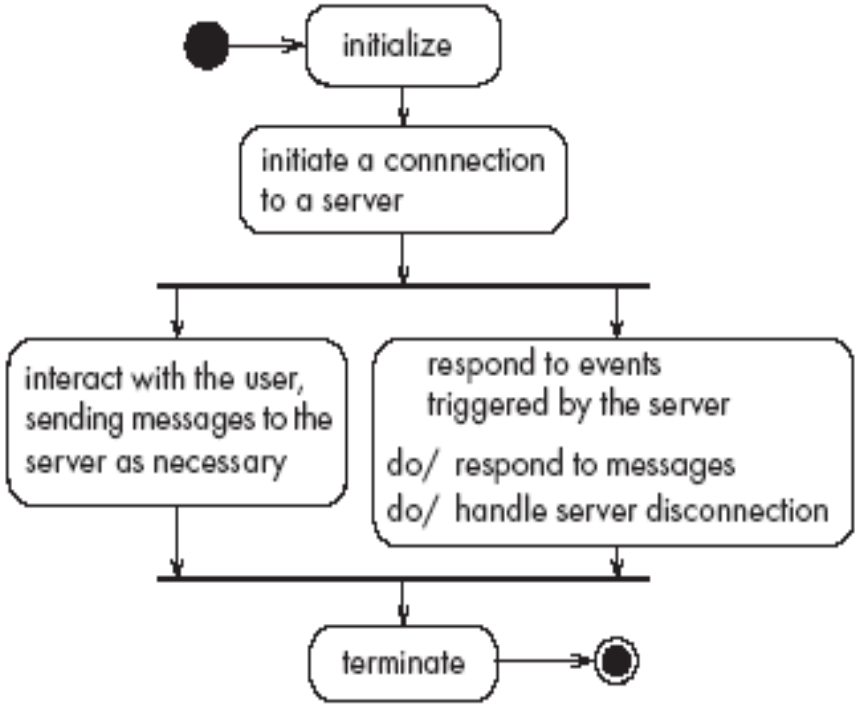
1. Initializes itself
2. Starts listening for clients
3. Handles the following types of events originating from clients
   A. accepts connections
   B. responds to messages
   C. handles client disconnection
4. May stop listening
5. Must cleanly terminate



Chapter 5- Structured Design

Activities of the Client:

1. Initializes itself
2. Initiates a connection
3. Sends messages
4. Handles the following types of events originating from the server
   A. responds to messages
   B. handles server disconnection
5. Must cleanly terminate



Chapter 5- Structured Design

Communications Protocols:

- The **messages the client sends to the server form a *language*.**
  - ✓ The server has to be programmed to understand that language.
- The messages the server sends to the client also form a language.
  - ✓ The client has to be programmed to understand that language.
- When a client and server are communicating, they are in effect having a conversation using these two languages
- The 2 languages & rules of conversation, taken together, are called *protocol*

Tasks to perform to develop client-server applications:

1. Design the primary work to be performed by both client and server
2. Design how the work will be distributed
3. Design the details of the set of messages that will be sent
4. Design the mechanism for:
   A. Initializing
   B. Handling connections
   C. Sending and receiving messages
   D. Terminating

Chapter 5- Structured Design

Technology Needed to Build Client-Server Systems/Software:

- Internet Protocol (IP)
  - **Route messages** from one computer to another
  - **Long messages** are normally split up into small pieces
- Transmission Control Protocol (TCP)
  - **Handles *connections*** between two computers
  - Computers can then exchange many IP messages over a connection
  - Assures that the messages have been satisfactorily received
- A host has an *IP address* and a *host name*
  - **Several servers can run on the same host**.
  - Each server is identified by a port number (0 to 65535).
  - To initiate communication with a server, a client must know both the host name and the port number; port number helps to specify the software that should receive the data (e.g., 80 for http).

Chapter 5- Structured Design

5.4. Interface Design :

- A ***user interface (UI)***: part of the system with which the users interact. It has 3 components: **navigation mechanism-** the way in which the user gives instructions to the system & tells it what to do (e.g., **buttons, menus); input mechanism-** the way in which system captures info (e.g., **forms** for adding new customers); **output mechanism-** the way in which system provides info to user (e.g., **reports).**
- Tech evolution enabled to move from: **text-based UIs** to **GUIs with windows**, **menus, icons, & a mouse**, to small **screen touch-based** mobile device apps.
- The study of **HCI** focuses on improving the interactions b/n users & computers by making computers more usable & receptive to user's needs.
- Mobile, ubiquitous, & social computing apps have broadened our concerns to that of the overall **user experience (UX)—**where users' feelings, motivations, & values are considered as well as efficiency, effectiveness, & satisfaction. A full discussion of UX design is beyond scope of this course; but, we will discuss usability-key aspect of UX.
- **system interfaces (SI):** help the **exchange of info b/n systems.** They are typically designed as **part of a systems integration effort-> *Middleware***

Chapter 5- Structured Design

5.4. Interface Design :

5.4.1. The Usability Concept:

- The term **usability** encompasses two related concepts: systems that are *easy to use* and *easy to learn*.
- A usable interface: **reduce user effort, enabling user to focus attention on the task at hand, not on making the system work**; usability **increases speed** of task completion & **reduces errors**.
- Usability: an **essential aspect of a quality IS**, **but developing a UI that ranks high** on **usability** **is neither a simple nor a trivial process**.

5.4.2. Principles for User Interface Design

- UI design is more of an art. Analysts must ***balance the need for simplicity*** & pleasant appearance **against the need to present the info across multiple pages or screens**, which decreases simplicity.
- Fundamental interface design principles, which are common for navigation, input design, & output design are given on table, next slide.

Chapter 5- Structured Design

5.4. Interface Design : …

Principles of User Interface Design (Dennis, Wixon & Roth, 2019, p.270)

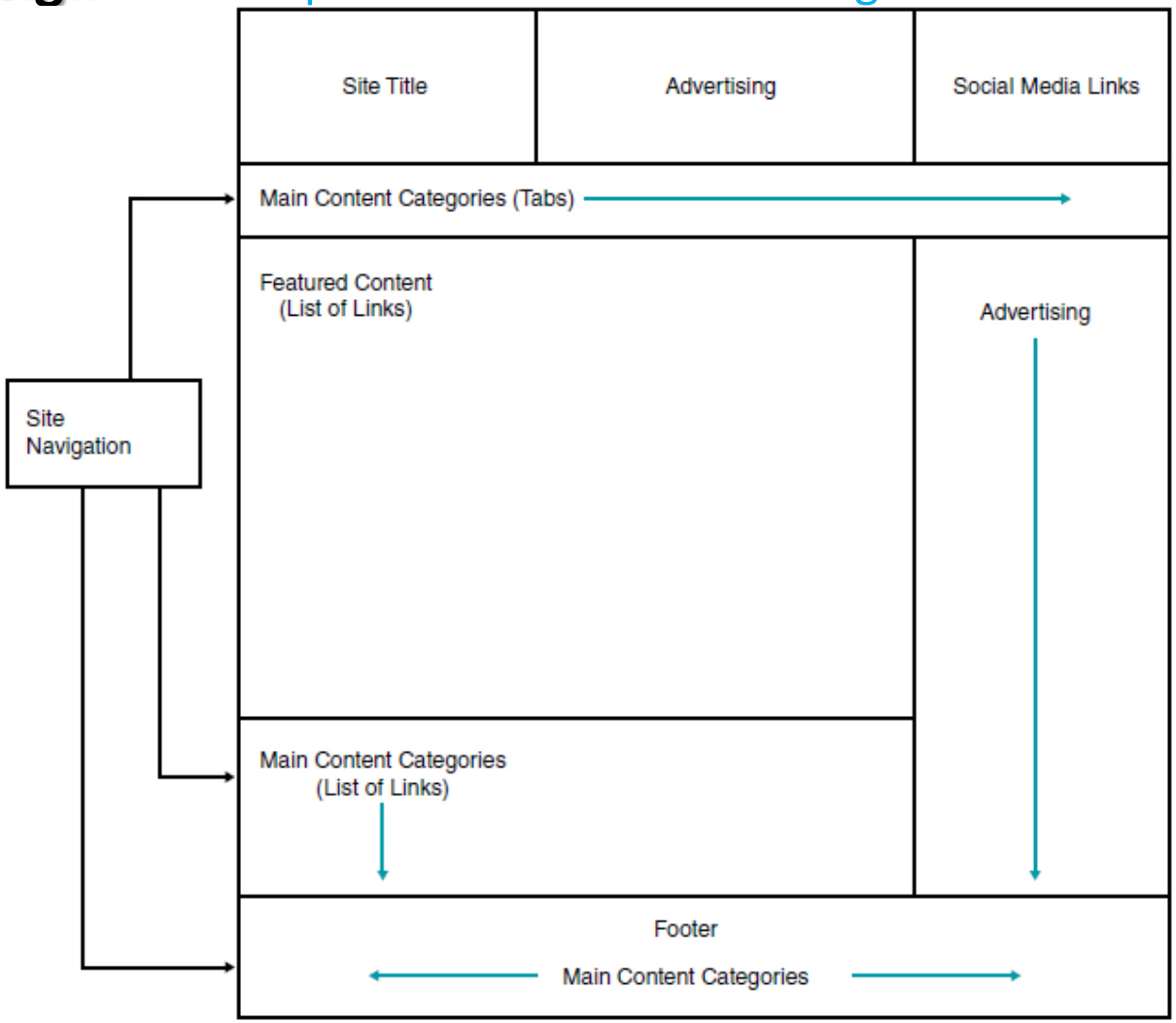
Principle	Description
Layout	The interface should be a series of areas on the screen that are used consistently for different purposes—for example, a top area for commands and navigation, a middle area for information to be input or output, and a bottom area for status information.
Content awareness	Users should always be aware of where they are in the system and what information is being displayed.
Aesthetics	Interfaces should be functional and inviting to users through careful use of white space, colors, and fonts. There is often a trade-off between including enough white space to make the interface look pleasing and losing so much space that important information does not fit on the screen.
Usage level	Although ease of use and ease of learning often lead to similar design decisions, there is sometimes a trade-off between the two. Infrequent users of software will prefer ease of learning, whereas frequent users will prefer ease of use.
Consistency	Consistency in interface design enables users to predict what will happen before they perform a function. It is one of the most important elements in ease of learning, ease of use, and aesthetics.
Minimize user effort	The interface should be simple to use. Most designers plan on having no more than three mouse clicks from the starting menu until users perform work.

Chapter 5- Structured Design

5.4. Interface Design : … Principles of User Interface Design

Web Page Layout

(Dennis, Wixon & Roth, 2019, p.271)



Model Web page layout.

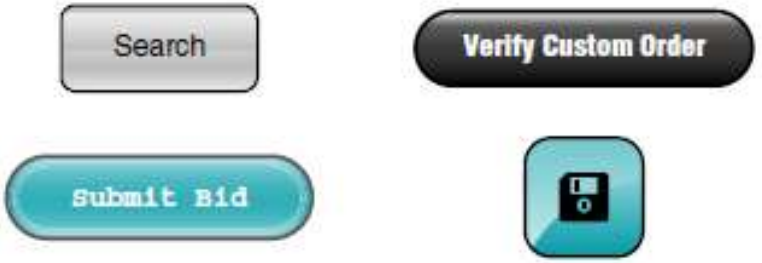
Chapter 5- Structured Design

5.4. Interface Design : … Principles of User Interface Design



Content awareness with menu selection highlights & ***breadcrumbs***-a *listing of the page hierarchy* (Dennis, Wixon & Roth, 2019, p.272)

Inconsistent symbols and terminology (Dennis, Wixon & Roth, 2019, p.275)



- Verify customer order/submit bid

vs

-Search what?

'Save'[see floppy disk symbol] what? -Size of buttons?

-

Chapter 5- Structured Design

5.4. Interface Design : …Special Issues of Touch Screen Interface Design:

- Keyboard & mouse have ruled the user interaction domain for decades, but recent advances in multi-touch technology have changed the game.
- **Apps involving significant data entry are not ideal for touch screen environments.**
- **Optimal use of touch screens is to retrieve/display info instead of enter new data.**
- Place **navigation elements at bottom** of the screen with **results/display area at the top. =>** This is to *avoid fingers from obscuring some of the content on the screen*.
- **Labels that describe a screen element should be along the top** of the element rather than the bottom so that *the finger does not hide the label* from the user
- bright background color/ pattern is preferred to hide glare and **reduce the annoyance of fingerprints on the screen**.
- Whenever possible, **use graphical objects that can be manipulated rather than entering text** (e.g., use a list box to scroll through a set of values).

Hand gestures are used to directly interact with interactive elements on the screen:

- ✓ It is *no longer possible to provide info to the user through hovering* & **tool tips.**
- ➢ important to use standardized gesture interactions to enhance the user's ease of learning and ease of use.

Chapter 5- Structured Design

5.4. Interface Design : …Special Issues of Touch Screen Interface Design:

- ✓ **Carefully consider which gestures are incorporated into the device where your application will be deployed & design appropriately** as there may be **differences between Windows, Apple, Android, and Blackberry devices**. Figure, below summarizes some of the more common Android touch screen hand gestures

Common set of gestures supported by Android devices (Dennis, Wixon & Roth, 2019, p.277)

Gesture	Meaning/Function
Navigational Gestures: Help users to move through a product easily. Supplement other explicit input methods, like buttons and navigational components	
Tap	Users can navigate to destinations by touching elements
Scroll and pan	Users can slide surfaces vertically, horizontally, or omnidirectionally to move continuously through content
Drag	Users can slide surfaces to bring them into and out of view
Swipe	Users can move surfaces horizontally to navigate between peers, like tabs
Pinch	Users can scale surfaces to navigate between screens
Action Gestures: Perform actions or provide shortcuts for completing actions	
Tap or Long press	Users can interact with elements and access additional functionality
Swipe	Users can slide elements to complete actions upon passing a threshold
Transform Gestures: Transform an element's size, position, and rotation with gestures	
Double tap or Pinch	Users can zoom into and out of content
Compound gestures	Users can fluidly transition between various gestures
Pick up and move	Users can reorder content with a long press and drag

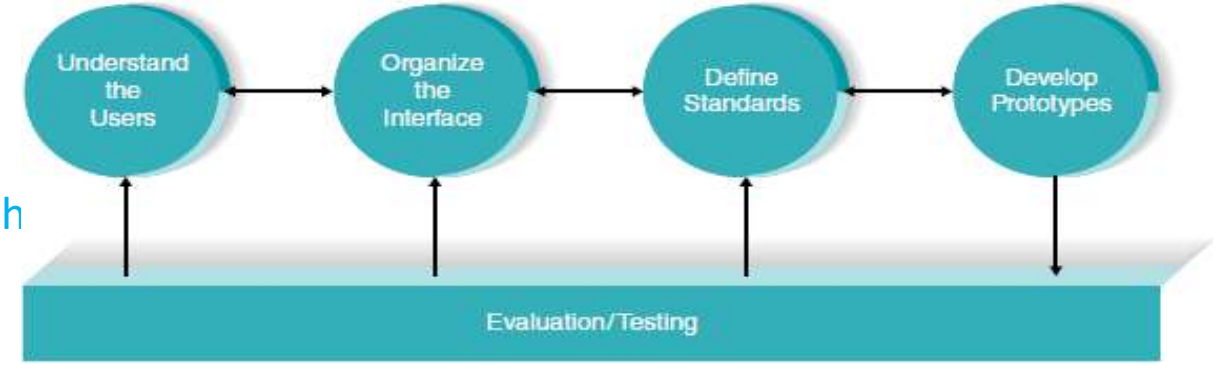
Source: material.io/design/interaction/gestures.html#types-of-gestures

Chapter 5- Structured Design

5.4. Interface Design…

5.4.3. User Interface Design Process: is 5-component iterative process analysts often move back & forth b/n components rather than proceed sequentially.

Interface Design Process (Dennis, Wixon & Roth 2019, p.278)



1st, *understand system users*, mainly if users are external to organization. Examine DFDs & use cases (will be discussed next chapter) from analysis phase to clarify users. User groups can be represented with **personas (characters representing user groups.** Depict common system usage patterns with ***use scenarios***.

2nd, organize interface using tools, like **interface structure diagram (ISD)** & **site map.**

3rd, analysts define **interface standards.**

4th, analysts create an **interface design prototype:** for navigation controls, input screens, output screens, forms (including preprinted paper forms), and reports.

5th , conduct **interface evaluation/testing** to determine whether they are satisfactory and how they can be improved.

Chapter 5- Structured Design

5.4. Interface Design…

Sales System Personas (Dennis, Wixon & Roth, 2019, p.279)

Martin: wants to learn details about DrōnTeq's products	
"I'm focused on finding the best commercial drone for my business needs."	
Knowledge:	Understands commercial drones and has specific uses and capabilities in mind for the drone purchase.
Tasks:	Arrives a site to select and configure custom drones. Wants to be able to easily identify drone models and options to meet his intended purposes. Wants to save and compare several configurations in terms of capabilities and prices.
Interests:	Has a business interest in commercial drones. Wants to find the best tool to use to fulfill business needs. Cost/benefit analysis is important.
Characteristics:	Deliberate; thoughtful; expects to be able to develop and compare alternative configurations easily and seamlessly.
Jesse: wants to learn about the uses of commercial drones	
"I'm intrigued by the possibilities of commercial drones and want to learn more."	
Knowledge:	Very limited specific knowledge of commercial drone uses and capabilities. Most likely knowledgeable about hobby-level drones. Wants to explore through examples, case studies, video clips, and owner descriptions of commercial drone applications.
Tasks:	Arrives at site and looks for video clips of drones performing commercial operations: photography, agricultural surveys, pipeline inspections, security surveillance, etc. Within each usage category, wants to discover more details through product descriptions, examples, and owner comments.
Interests:	Has a general interest in commercial drone applications and prefers to see an array of ideas and examples.
Characteristics:	Curious, open-minded; creative, exploratory. Wants to get excited by the possibilities offered by commercial-level drones.

DrōnTeq Sales System personas.

Chapter 5- Structured Design

5.4. Interface Design…

Two Use Scenarios for the Drone Teq Sales Website
(Dennis, Wixon & Roth, 2019, p.280)

Use Scenario: Focused Visitor
User wants to easily develop alternative drone configurations and compare price and performance of the alternatives
1. User selects a drone base model 2. User looks at lists of available options for that model 3. User selects options and wants to learn price and capabilities of that configuration 4. User saves, modifies, or discards that configuration and repeats with other options or other drones 5. User has one or more configurations to save and consider for purchase. 6. User may want to contact a sales representative with questions.

Use Scenario: Exploratory Visitor
User wants to learn about applications of commercial drones through video clips and written material
1. User views video clips of drones being used in commercial applications 2. User selects an application area and views more clips in that area 3. User reads site marketing material 4. User reads owner descriptions and comments 5. User repeats steps 2-4 for other commercial application areas. 6. User views list of drone models 7. User selects a model and reads site marketing material for that drone.

Chapter 5- Structured Design

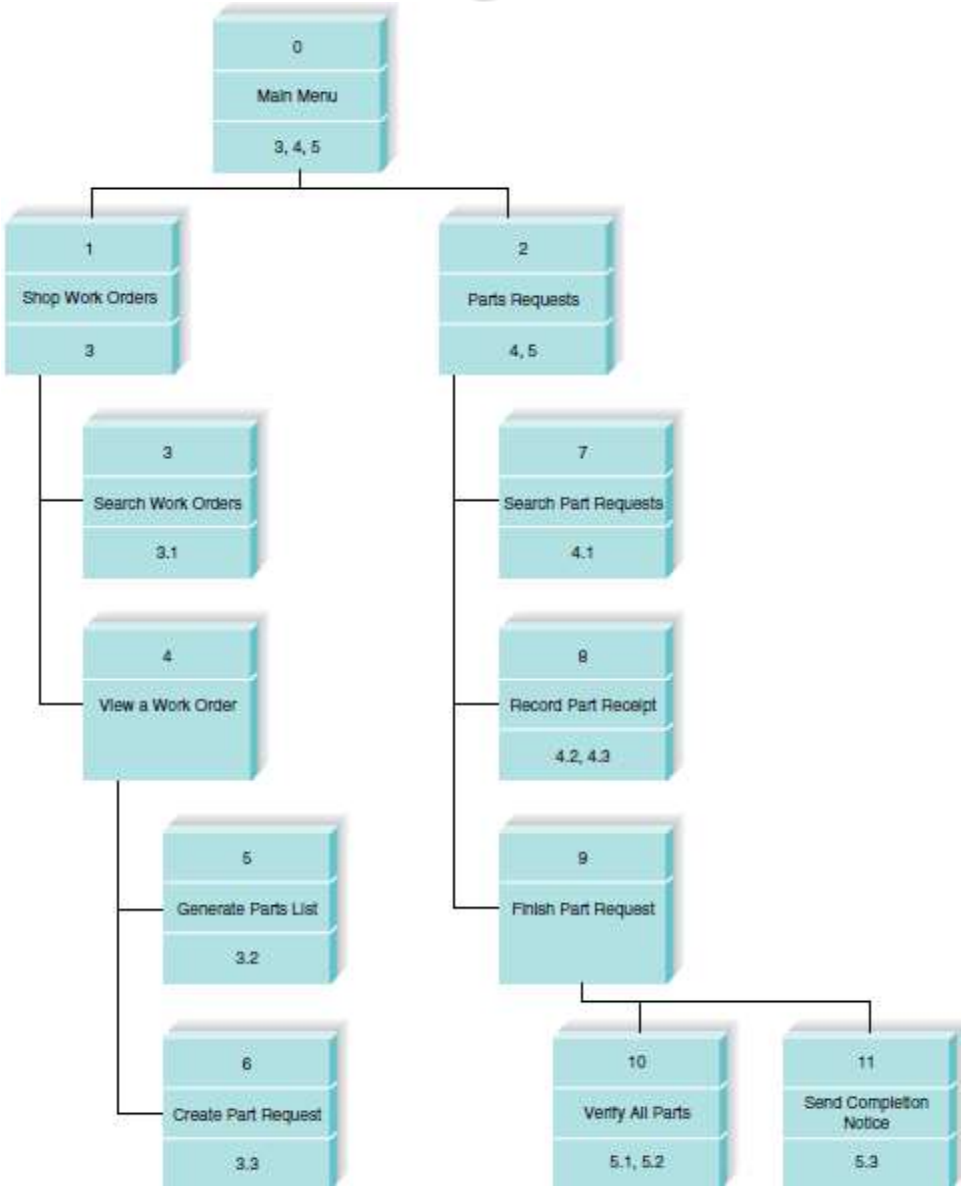
5.4. Interface Design…

Shop management interface structure diagram (Dennis, Wixon & Roth, 2019, p.281)

- An *Interface structure Diagram is used to show* how all the screens, forms, & reports used by the system are related & how user moves from one to another.

Most systems have several ISDs, one for each major part of the system.

- Draw **each element (e.g., screen, form, report) as a box** & give it a unique # (at top) # a unique name (in the middle).
- Nos show r/ship b/n menu & Submenus. The nos do not mean that all the screens belong to "parents" higher in the tree.
- Lines denote ability to navigate from one menu to another

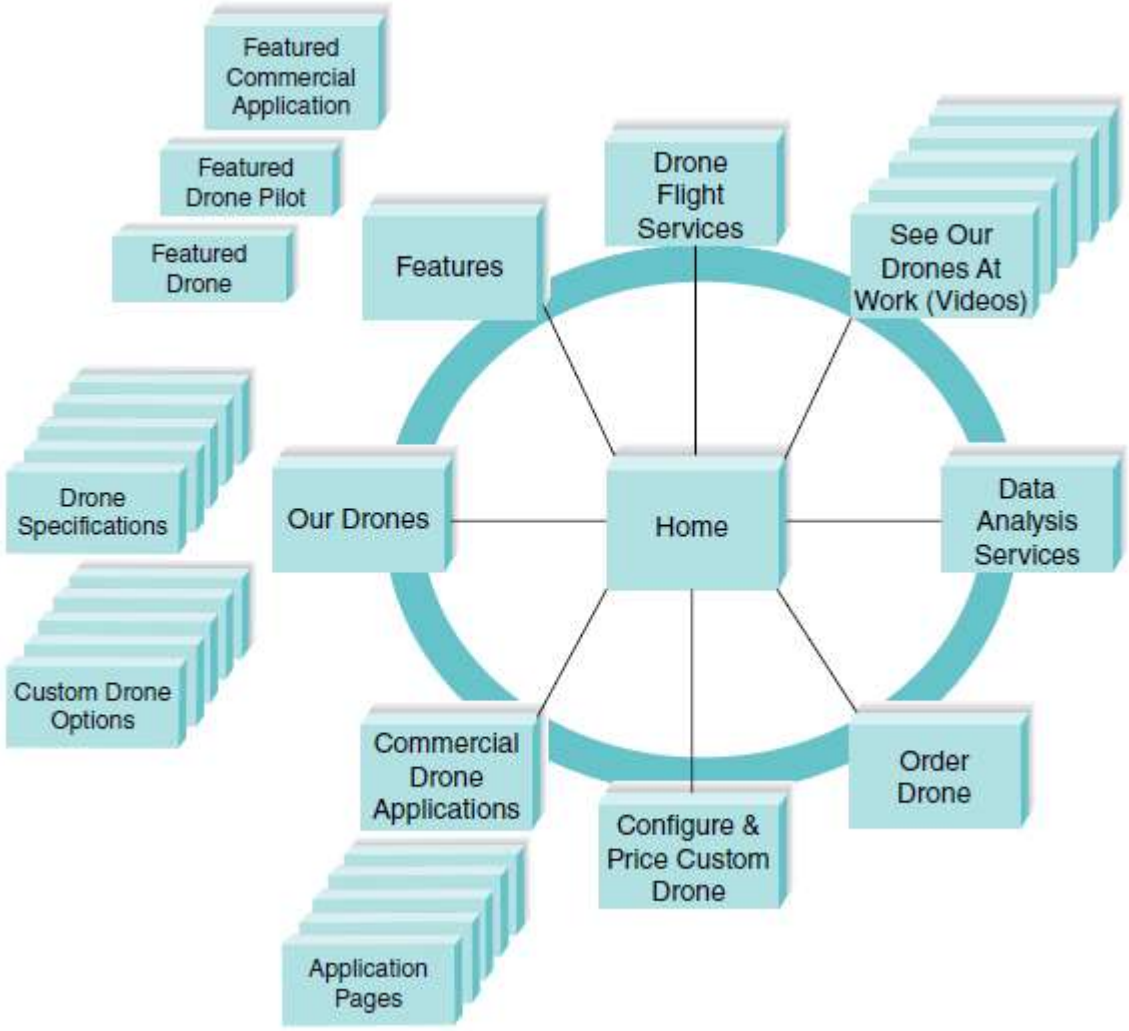


Chapter 5- Structured Design

5.4. Interface Design…

Site map (Dennis, Wixon & Roth, 2019, p.282)

- A site map helps analysts **clarify how all the info on the site fits together** & helps establish the hierarchy of info on the site.
- There is **no standard format for a site map**: some are drawn as a hierarchy while others look more like a network of nodes.



Chapter 5- Structured Design

5.4. Interface Design…

INTERFACE STANDARDS: basic design elements that are common across individual screens, forms, and reports within the system.

- ❑ Depending on app., there may be several sets of interface standards for d/t parts of system (e.g., one each for Web screens, paper reports, input forms).
- ❑ ***Standards***: serve as **touchstone *ensuring that interfaces are consistent*** across syst.

1. **Interface Metaphor**: **concept from real world used as a model** for the info sys.

- ❑ **Metaphor** helps user to **understand system**. E.g. **Cart for e-commerce sites**.

2. **Interface Objects and Actions:** use terminology that fits user expectations throughout.

- ✓ **Interface objects-** fundamental building blocks of sys.: like entities, data stores.

  E.g., **choose one & use consistently: buyers vs customers**

- ✓ **Interface actions**- common commands employed by users.

  E.g., **choose one & use consistently: Buy** vs **Purchase**.

3. **Interface Icons:** used to **represent interface objects, actions & their status** (e.g, deleted, err). Example: **pencil indicates edit**; **trashcan indicates delete**.

Chapter 5- Structured Design

5.4. Interface Design… INTERFACE STANDARDS….

4. **Interface Templates: define the general appearance** of all interface components (screens, forms, & reports).

Template design: **specifies the basic layout of screens**:

e.g., navigation area(s), status area, & form/report area(s);

color scheme(s) to be applied, & font styles & sizes to be used.

- ✓ It defines whether windows will:
  - ❖ replace one another on the screen or
  - ❖ **cascade on top of each other**.
- ✓ Template **defines standard placement** & order for common interface actions.

  e.g., **"File, Edit, View"** rather than

  "File, View, Edit"

- ➢ In short, the **template draws together all the other major interface design elements**: **metaphors, objects, actions, and icons**.
- ➢ Templates: help ***ensure user interface consistency throughout the system***.

Chapter 5- Structured Design

5.4. Interface Design…

5.4.4. **Navigation Design**: The navigation component enables user to move throughout the system, perform actions, and receive feedback messages as needed. The **goal of the navigation system** is to make the system as simple as possible to use

Basic Principles of Designing Navigation Controls include: **prevent mistakes**, simplify **recovery from mistakes**, **use consistent grammar order**, proper **menu & message tips**.

5.4.5. **Input Design**: means **designing screens used to enter info**, as well as any forms on which users write or type info (e.g., time cards, expense claims).

- Input mechanisms facilitate entry of data into the computer system, whether highly **structured data**, such as order info (e.g., item numbers, quantities, costs), or **unstructured info** (e.g., comments).

Basic Principles Designing Input: Use **Online (real-time)** & **Batch Processing** Appropriately, **Capture Data at the Source-** i.e., employ **source data automation** (barcode readers, OCR, magnetic strip readers, smart cards & radio frequency identification-RFID-tag), **minimize keystrokes**(retrieving from DB, select from list, default value), Input tips(e.g., field labels). **Input Validation/edit checks/**: completeness check, format check(e.g., numeric /date), range check (e.g, min & max values), check digit check(nos. added to a code-e.g., 9th digit to US SS no- as a way of enabling the system to quickly validate correctness), consistency check, & database check (check data against db for correctness).

Chapter 5- Structured Design

5.4. Interface Design…

5.4.5. Output Design

- Outputs: are **reports the system produces**(on screen, paper, or in other media-the Web). They are the most visible part of any system, because a **primary reason for using an info system is to access the info that it produces**.

Basic Principles of Output Design: ***understand report usage*** (sorting-A toZ/ZtoA, page numbers, dates, frequency-real time/batch reports), ***manage info load*** (avoid too much-for managers or too little, most important info 1st), ***minimize bias*** (when sorting, graphic reports).

5.4.6. Types of Output: ***detail reports*, summary reports, Exception reports,** ***turnaround documents** (outputs turned to input-e.g utility bills), and **graphs.**

5.4.7. Media: Many organizations produce reports electronically, whereby reports are "printed," but stored in electronic format on file servers or Web servers so that users can easily access them. **Electronic reports**-produced on demand & searched more easily for certain words, enable customized ad hoc reports generated when needed. Some users may still print the electronic report on their own printers. The reduced cost of electronic delivery over distance & improved user access to the reports usually offsets the cost of local printing.

*Form printed & sent for user to fill in; the filled in form is then encoded into system

Chapter 5- Structured Design

5.5. Software Reuse/Component /Design

- Software reuse is the practice of using existing software artifacts to build new software components.
- **Component design**: is a key technology in software reuse.

Some **tips for component design and software reuse**

- **Plan**: Before building a new component, check if it already exists & try to reuse it.
- **Collaborate**: Work with multiple SW engineers during design & definition phases.
- **Analyze**: Include a reuse analysis stage in detailed design work to assess opportunities for reuse.
- **Extract reusable elements**: Identify reusable elements in the application system.
- **Create a library**: **Use reusable elements to form a reusable component library**.
- **Select components**: When designing an application system, choose suitable components from the library & instantiate them.
- **Compose components**: Implement the software system by **composing the instantiated components under a reusable framework**.

Chapter 5- Structured Design

5.5. Software Reuse/Component /Design…

- Software reuse: is the **process of implementing or updating software systems** using **existing software components**.
  - The **systematic development of reusable components**
  - The **systematic reuse of these components as building blocks to create new system**
- **Reuse**: **building on the work & experience of others**
- SW engineers should ***avoid re-developing software already developed***
- Types of reuse:- => ***Reuse of***:
  - **Expertise**
  - **Standard designs & algorithms**
  - **Libraries of classes or procedures**
  - Powerful commands built into languages & operating systems
  - **Frameworks**
  - **Complete applications**

Chapter 5- Structured Design

5.5. Software Reuse/Component /Design…

Reusability and Reuse in SE:

❑*Reuse* & design for *reusability* should be part of the culture of SW Devt organs

❑But there are **problems to overcome**:

✓ Why take extra time to develop something that will benefit other projects/customers?

✓ Mgt may only reward efforts of people who create *visible 'final products'*.

✓ Reusable software is often created in a hurry and *without enough attention to quality*.

❑Developers tend not develop quality reusable components, so there is often little to reuse. **To solve the problem, recognize that**:

- This vicious cycle costs money
- ***Investment* in reusable code is important**
- **Attention to *quality* of reusable components is essential**
  - ✓ So that **potential re-users have confidence in them**
  - ✓ Quality of a SW product is **only as good as its lowest-quality reusable** component
- Developing ***reusable components can often simplify design***

Chapter 5- Structured Design

5.5. Software Reuse/Component /Design…

Frameworks: Reusable Subsystems

❑A *framework* is **reusable software that implements a generic solution to a generalized problem**.

- ✓ It **provides common facilities applicable to d/t application programs**.

❑ ***Principle***: *Apps that do d/t, but related, things tend to have similar designs*

Frameworks to promote reuse:

❑A framework is **intrinsically *incomplete*:**

- – Certain classes/methods are used by the framework, but are missing (*slots*):
  - ❖ The ***more slots*** that the app developer must fill, the **more complex** the framework is to use.
  - ❖ **More flexible**
- – Some functionality is optional
  - ❖ Allowance is made **for developer to provide it (*hooks or extension points*)**
- – **Developers use the *services* that the framework provides**
  - ❖ Taken together the **services are called Application Program Interface (*API*)**

Chapter 5- Structured Design

5.5. Software Reuse/Component /Design…

Object-oriented frameworks:

- In the **OO paradigm (chapters: 6-8), a framework is composed of a library of classes**.
  - The **API is defined by the set of all public methods of these classes**.
  - Some classes will be abstract & there are often many interfaces

E.g., A framework for Payroll /Univ. registration/e-commerce sites

Frameworks and product lines

- ***product line/product family:** is a ***set of products built on a common base of technology***.
  - The various products in the product line have **d/t features to satisfy d/t markets**
  - The **SW technology common to all products is included in a framework**
  - Each product is produced by **filling the available hooks and slots**

    E.g. software products offering 'demo', 'lite' or 'pro' versions

---

*** Product-line software:**

- Composed of reusable components and designed to provide specific capabilities for use by many different customers. It may focus on a limited and esoteric[unique] marketplace or attempt to address the mass consumer market (Pressman & Maxim, 2020, p.7).
- …. families of related software products sharing common features (Pressman & Maxim*, 2020, p.164).*
- A set of application programs that are built from a common set of reusable software modules designed to be easily adaptable when creating in new software products (Pressman & Maxim, 2020, p.590).

Chapter 5- Structured Design

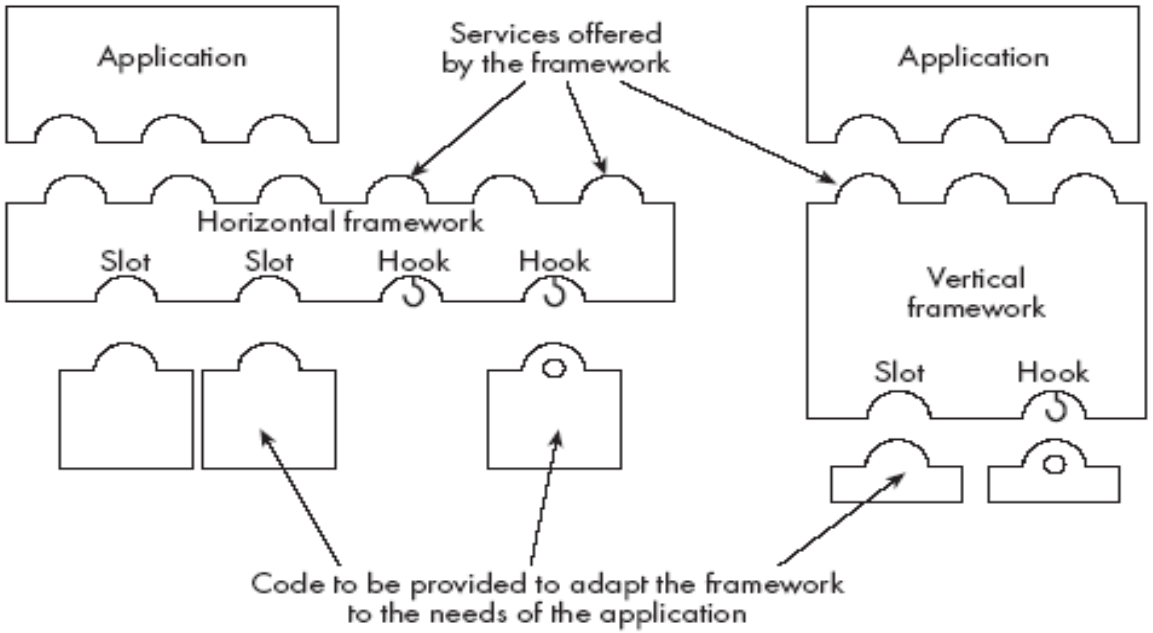
5.5. Software Reuse/Component /Design…

Types of frameworks:

- A ***horizontal*** framework provides general application facilities that a large number of applications can use.
- A ***vertical*** framework (*application framework)* is more 'complete' but still needs some slots to be filled to adapt it to specific application needs

Slot: missing parts (e.g., classes/methods) in a framework, in which the app **developer need to fill *in an app specific way to adapt the framework to specific needs*.**

Hooks: like slots but developer can add optional functionality



Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

- **Data design** (sometimes referred to as data architecting*)* creates a model of data and/or info. that is represented at a high level of abstraction (the customer or **user's view of data**).
- At the ***program-component level***, the *design of data structures* & the *associated algorithms* required to manipulate them is essential to the creation of high-quality applications.
- At the ***application level***, the *translation of a data model* (derived as part of requirements engineering) *into a database is pivotal to achieving the business objectives of a system*.
- At *the* ***business level***, the collection of info stored in disparate databases and reorganized into a "data warehouse" enables data mining or knowledge discovery that can have an impact on the success of the business itself. In every case, data design plays an important role.
- Because the most popular storage format today is the relational database, we briefly discuss on designing these databases.

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

- As explained in Section 5.2., the work done by any application program can be divided into four general functions: **data storage, data access logic, application logic, and presentation logic.**
- The ***data storage function*** is **concerned with how data is stored and handled by the programs that run the system**.
- It is essential to **ensure that data storage is designed so that inefficient systems, long response times, and difficult-to access info (several hallmarks of failed systems) are avoided**.
- Major ***tasks in data storage design***:
  1. **Select the data storage format** for the new system: two basic types of data storage formats for application systems are: **files & databases**
  2. **Convert the logical data model to physical data model**: ensure that the DFDs and ERDs balance properly; so the CRUD matrix will be revised as necessary
  3. ***Design selected data storage format*** **to optimize its processing efficiency.**

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

5.6.1. Data Storage formats…. Files

- Two main data storage formats: files and databases.

I. **Files:** are electronic lists of data, optimized to perform a particular transaction. Example, a **patient appointment file** (Appt date, time, duration, reason, patientID,Name, PhoneNo, DoctorID, Doctor Name), in the form in which it is used, so that it can be accessed and processed quickly by the system.

A ***data file:*** *contains an electronic list of info that is formatted for a particular transaction;* info is manipulated by programs that are written for those purposes.

Sequential access files*:* **allow only sequential file operations to be performed (e.g., read, write, and search**); very **efficient in:** writing or infrequent operations . But ***inefficient for random operations:*** **finding/updating a specific records**. They come in 2flavors: ordered & unordered.

- *Unordered sequential access file:* list of info stored on disk. They are organized serially (i.e., order of file is the order in which the records are written to the file); new objects simply are added to the file's end.
- *Ordered sequential access files: are placed into a specific sorted order (e.g., ascending/descending* by custID). There is overhead associated with keeping files in a particular sorted order.

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design… Files

- File designer **keeps file in sorted order** by:

1. **Always creating a new file for every delete/addition**,
2. A **pointer**: *info about the location of the related record. It* is placed at end of each record, & it "points" to next the record in a series/set.
   - **Underlying data/file structure in this case is the *linked list data structure***

Random access files (RAF): allow only random/direct file operations to be performed.

- Optimized for random operations: like ***finding & updating*** a specific records.
- RAF gives faster response time to find & update operations than any other file type.
- RAF doesn't support sequential processing (e.g., report writing); they are inefficient.

Legacy system files: are mainly in a proprietary format, while newer systems use a standard format such as CSV (comma separated value)/tab-delimited.

Types of files : based on the way they are used to support an application: **master files**, **look-up files**, **transaction files**, **audit files**, and **history files**.

Master files: store core info about the bus. application, such as order or customer.

- Kept for long periods, & new records are appended to the end of file as new orders are captured by system.
  - To change existing records, programs must be written to update old info.

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design… Files

Look-up files: contain static values (like list of valid codes or names of countries).

- Typically, list is used for validation. E.g., if a customer's mailing address is entered into a master file, the **country name is validated against a look-up file** that contains countries to make sure that the value was entered correctly.

Transaction file: holds info that can be **used to update a master file**.

- These files **can be destroyed after changes are made**, or the file may be saved in case the transactions need to be accessed again in the future. Example:- Customer address changes would be stored in a transaction file until a program is run that updates the customer address master file with the new information.

Audit file: records "before" & "after" images of data as the data are altered, so that an audit can be performed if the integrity of the data is questioned. E.g.: As human resources clerk changes employee salaries in HR system, system should record the person who made the changes to the salary amount, date, & actual change made.

History file (or archive file): stores **past transactions** (e.g., inactive customers, past orders) that are no longer needed by system users.

- File is stored off-line, yet it can be accessed as-needed. Other files, such as master files, can then be streamlined to include only active or very recent information

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

5.6.1. Data Storage formats…. Databases

II. A **database:** info that are related to each other (e.g., through common fields). E.g., **customer, order**, & **product** information related logically.

- A **DBMS:** is software that creates and manipulates these databases.
- **End-user DBMS:** support small-scale databases to enhance personal productivity (e.g., MS Access); less expensive & easier for novice user; can't support mission critical/large systems
- **Enterprise DBMSs:** can manage huge volumes of data & support applications that run an entire company. E.g., DB2, SQL Server, & Oracle; expensive but support large/mission systems
- **Open-source DBMS's**: are also popular, such as MySQL.

Main types of databases:

- **Legacy, Relational,**
- **Object,**
- **Multidimensional , NoSQL**.

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

5.6.1. Data Storage formats…. Databases

1. Legacy Databases: not common today. E.g.: Hierarchical & Network model

Hierarchical databases (e.g., IDMS) use hierarchies, or inverted trees, to represent r/ships (similar to the 1:M r/ship).

- ❑ They are known for rapid search capabilities, but can't efficiently represent many-to-many (M:N) r/ships or nonhierarchical

Network databases (e.g., IDMS/R, DBMS 10) records are related to each other through **pointers.** **It can represent M:N relationships**

- ❖ Both legacy systems handle data efficiently, but require a great deal of programming effort; application programs must understand how the database is built & must be written to follow the structure of the database.

2. Relational database: most popular kind of database for app dev't today.

- ❑ *Less "machine efficient"* than its legacy counterparts, but *much easier to work with from a dev't perspective*.
- ❑ Based on collections of tables, each with a **primary key(PK)**—a field(s) whose value is d/t for every row of the table.

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

5.6.1. Data Storage formats…. Databases

- Tables are related to each other via a primary key from one table into the related table as a foreign key(FK).
- RDBMSs: support **referential integrity-** idea of ensuring that values linking tables through primary & foreign keys are valid and correctly synchronized. E.g., There won't be an order w/o reference to custID from Customer table.
- **Struct. Query Lang (SQL):** standard language for accessing data in tables.
- Queries on multiple tables, 1st join the tables together on the basis of their PK & FK r/ships & treated as if they were one large table. E.g.: MS Access, Oracle, DB2, Sybase, Informix, MS SQL Server, & MySQL.

3. Object /Object-Oriented Databases: basic premise of OO: all things should be treated as objects, having both data (attributes) & processes (behaviors).

- **OODBMSs: mainly used to support** multimedia apps/systems that involve complex data (e.g., graphics, video, and sound). Telecoms, financial services, health care, & transportation have been the most receptive to object databases. Overall, OODBMSs play a minor role in the DBMS market.

  E.g., Gemstone, Objectivity, db4o, and Versant

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

4. Multidimensional databases (MDDBs):

- Type of **relational database, used extensively in data warehousing**.
- **Data warehousing: practice of taking data from a company's transaction processing systems, transforming it (e.g., cleaning up, totaling, aggregating them)**, & then **storing the data for use in a data warehouse** (i.e., a large database) that supports **business intelligence (BI) systems**.
- A **data warehouse**: usually relies on relational technology as its storage format; however, companies can create **data marts,** which are **smaller databases based on data warehouse data**. Typically, a **data mart** receives **downloads of data from the data warehouse regularly**, and it supports ***BI for a specific department or functional area*** of the company.
- MDDBs allow a user to ask questions like "How many Toyota V8 automobiles have been sold in Ethiopia so far this year?" It helps to get info that is **aggregated** (e.g., totaled or averaged) across many records (e.g., "What was the average sales by quarter for product A?").

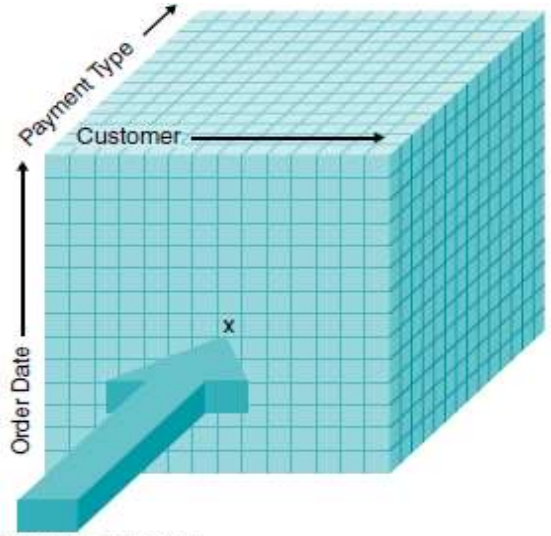
➢ Data marts: require that data be stored in a format in which they can be easily aggregated & manipulated across a variety of dimensions (e.g., time, product, region, sales rep).

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

- Legacy, object, & relational databases are not designed to store data to support aggregations of data on multiple dimensions.
- When data are 1st loaded into a MDDB, it pre-calculates the data across multiple dimensions & stores answers, using arrays /other technique.
- **Initial loading of data can be quite slow b/c of all calculations** that must take place; But data access is fast b/c "answers" already exist in the arrays.

Example:- the cube in Figure represents a MDDB containing data that have been organized by customer, payment type, & order date. Pre-calculated quantitative info (e.g., totals, averages) is stored at intersection of the dimensions (in each block), & the BI app directly accesses those blocks.



Last quarter, how many customers placed more than one order, using an American Express card?

Dennis, Wixon & Roth, 2019, p.361

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

5.6.1. Data Storage formats…. Databases

5. NoSQL Databases

- **Relational model was created/appropriate when storage was limited;**
- No longer necessary to break an organization's data up into many tables.
- Today organizations want to store audio, video, and image files which do not fit well in a relational structure.
- Hence, **NoSQL databases** emerged: they contain a wide variety of database approaches that evolved for different purposes. The primary common theme is that **the relational model and SQL are not used in these systems**.

4.1. ***Document-oriented databases:***

- Manage collections of docs where those docs *can* have a variety of structures including large bit files for image, audio, & video data.
- No need for uniform record structure: diff. records may have diff. columns.
- The types of the values of individual columns can be different for each record, and columns can have more than one value (arrays).

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

4.1. ***Document-oriented databases:*** Records can also have a nested structure.

Example: Mongo DB is an open-source, document-oriented database

4.2. ***Wide column stores:*** also called extensible record stores

- ❑ **Store data in records** with an ability to hold **very large numbers of dynamic columns** (potentially **billions of columns**). Examples:
- ❖ Google's Bigtable: considered the origin of this category of databases.
- ❖ Cassandra: was created by Facebook. Now (become open source in 2008), Apache has dubbed it a Top Level Project (TLP), giving it a high degree of respectability among open source projects.
- ❖ Amazon also developed its own database called Dynamo.

4.3. ***Graph databases:*** use graph theory to store, **map**, and **query** relationships.

- ❑ Essentially a collection of nodes & edges:
- ❖ Each node represents an entity. Each edge represents a connection or r/ship b/n two nodes. Suitable for analyzing interconnections & are useful when mining data from social media. E.g., Amazon's Neptune
- ❖ When you encounter the feature "Customers who bought this also looked at . . ." on websites, you've probably encountered use of a graph database

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

- NoSQL databases support very high transaction rates processing relatively simple data structures, replicated on many servers in the cloud.
- The **market for DBMSs**, however, is still **heavily dominated by relational DBMSs,** mainly due to the strength of Oracle, MS SQL Server & MySQL.
- A recent popularity ranking of DBMSs found that top four were relational.
- Out of remaining top 10, 2 were relational & 4 four were NoSQL.
- ➢ It seems that NoSQL databases are becoming more popular.

Selecting a Storage Format

- Each of the file & database data storage formats has its strengths & weaknesses, and no one format is inherently better than the others.
- Sometimes, a team may choose multiple formats (e.g., a relational db for one data store, a file for another, & a multidimensional db for a 3rd).
- ➢ Thus, it is important to understand the strengths and weaknesses of each format and when to use each one

Chapter 5- Structured Design

Selecting a Storage Format…

	Files	Legacy databases	Relational databases	Object databases	Multidimensional databases	NoSQL databases
Major strengths	Files can be designed for fast performance; good for short-term data storage.	Very mature products	Leader in the database market; can handle fast updating and querying needs	Able to handle complex data	Configured to answer business intelligence questions quickly	Designed for huge, varied data sets
Major weaknesses	Redundant data; data must be updated using programs.	Not able to store data as efficiently; limited future	Cannot handle complex data	Limited market acceptance; skills are hard to find.	Highly specialized use; skills are hard to find	New in the market, highly specialized use; skills are hard to find
Data types supported	Simple	*Not recommended for new systems*	Simple	Complex (e.g., video, audio, images)	Aggregated	Mixed data sets with structured and unstructured components
Application system types supported	Transaction processing	*Not recommended for new systems*	Transaction processing and decision making	Transaction processing	Business intelligence	Business intelligence; finding patterns and relationships in mixed data
Existing data formats	Organization dependent	Organization dependent	Organization dependent	Organization dependent	Organization dependent	Organization dependent
Future needs	Limited future prospects	Poor future prospects	Good future prospects	Uncertain future prospects	Uncertain future prospects	New, uncertain future prospects

Comparison of data storage formats.

Dennis, Wixon & Roth, 2019, p.363

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

Physical Entity Relationship Diagrams-ERDs:

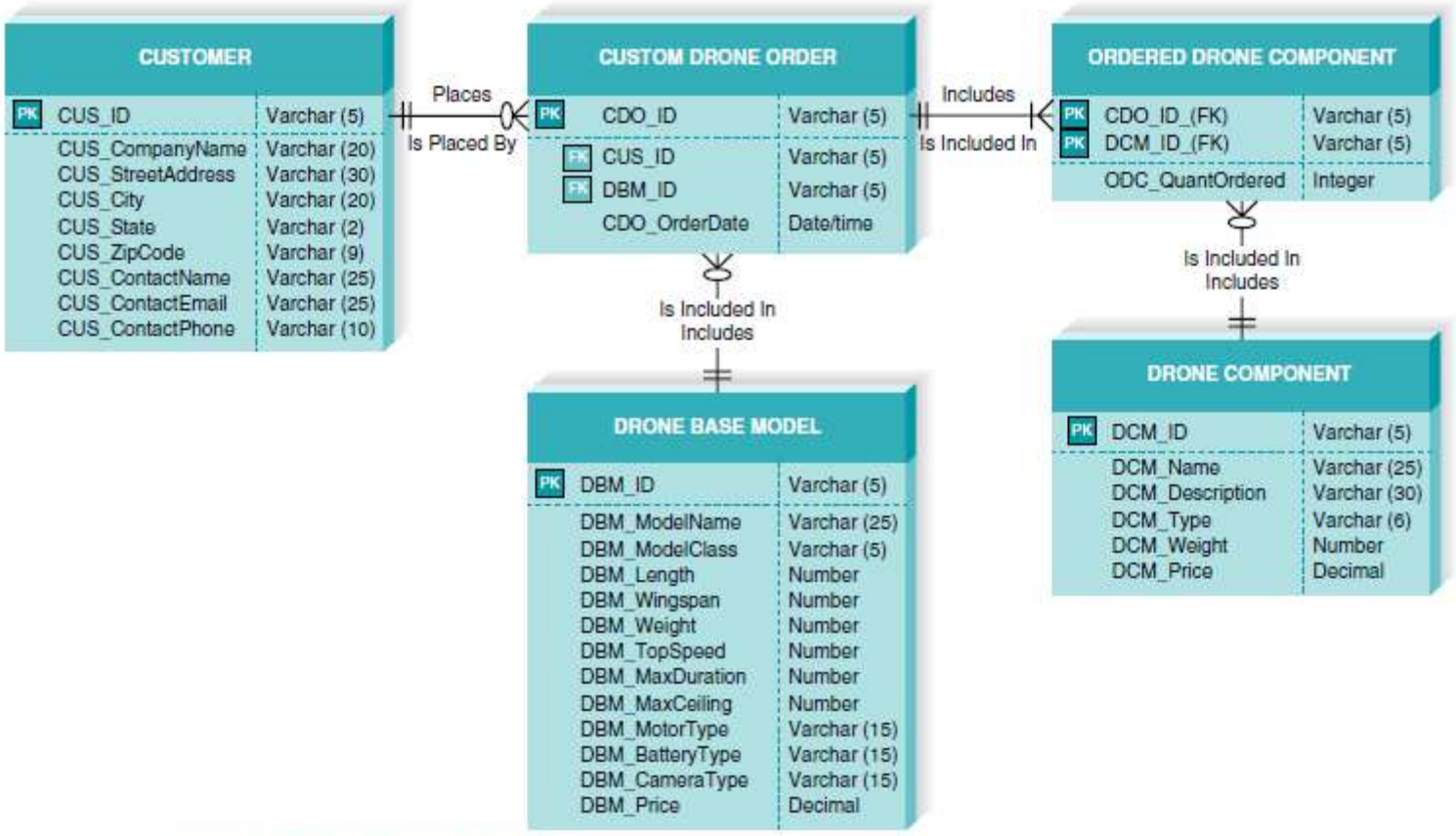
- ERD contains same components for both logical &physical models including entities, relationships, and attributes.
- The difference lies in the fact that *physical ERDs* contain references to exactly how data will be stored in a file/ db table
- Steps to transition from logical to physical data model is fairly straightforward:

Step	Explanation
1. Change entities to tables or files	Beginning with the logical ERD, change the entities to tables or files and update the metadata
2. Change attributes to fields	Convert attributes to fields & update the metadata
3. Add primary keys	Assign primary keys to all entities
4. Add foreign keys	Add foreign keys to represent the r/ships among entities
5. Add system-related components	Add system-related tables and fields

Chapter 5- Structured Design

5.6. Data Structure /Data Storage Design…

Physical Entity Relationship Diagrams-ERDs:



Custom drone ordering system physical ERD.

Dennis, Wixon & Roth, 2019, p.366

Chapter 5- Structured Design

5.7. Algorithm/Program Design

Program design: part of design phase in which analysts determine programs to be written, create instructions for programmers about what the code should do, & identify how pieces of code will fit together to form system.

Steps:

1st, **Choose programming language**

2nd, **Revise/convert logical DFDs into physical DFDs** is outlined.

3rd , **Organize the processes** using a **structure chart** to **depict their decisions.**

4th**, develop** detailed instructions called **program specification:** so that programmers know exactly what they should be creating.

Together, structured chart & program spec communicate how the application logic for the system needs to be developed.

- **Create physical process models to show implementation details** and explain how the final system will work.
- These details can include references to actual technology, the format of info moving through processes, & the human interaction that is involved.

Chapter 5- Structured Design

5.7. Algorithm/Program Design

❑These to-be models describe characteristics of the system that will be created, communicating the "systems view" of the new system

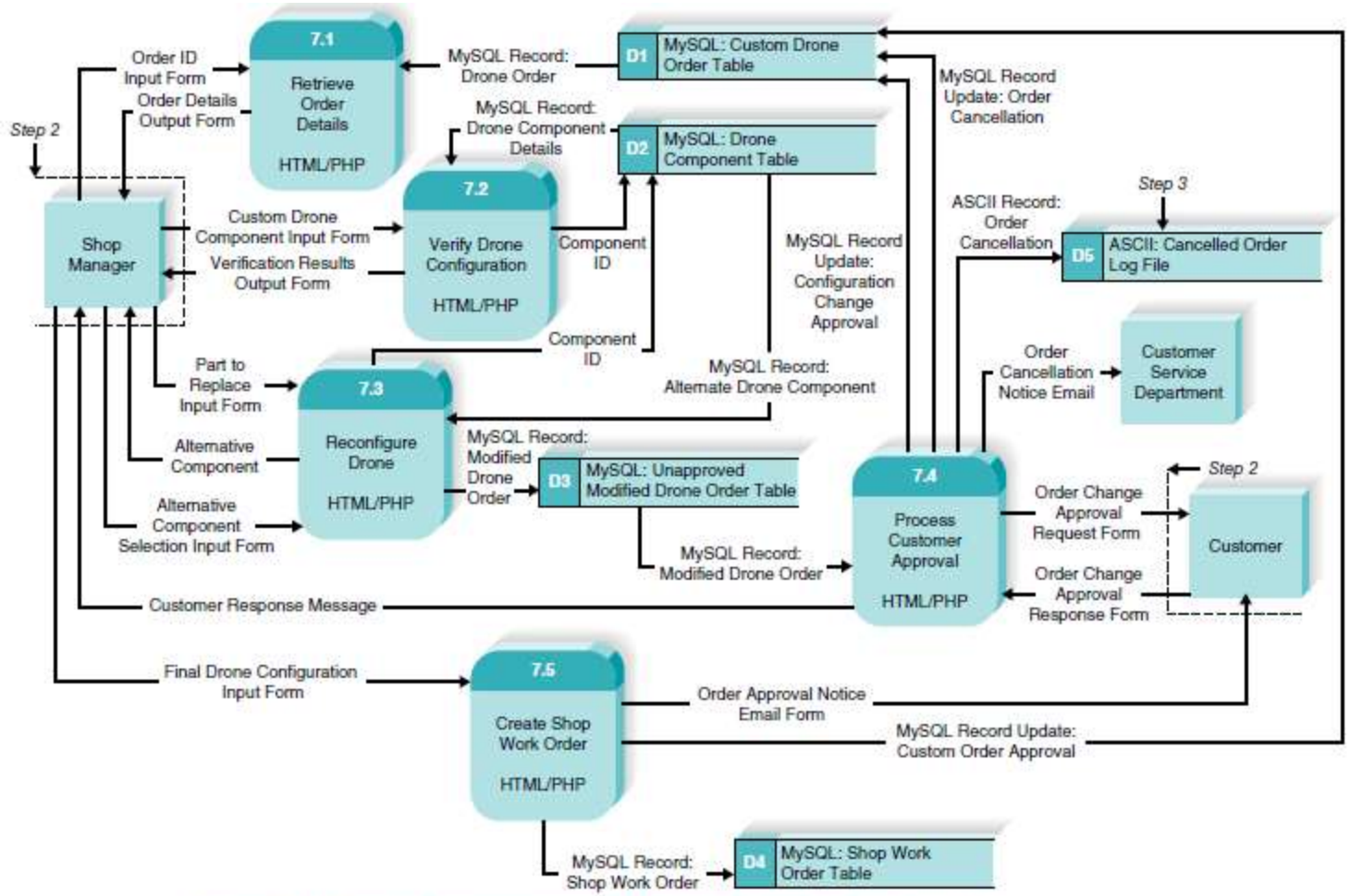
Physical DFD: contains same components as logical DFD (e.g., data stores, data flows), & same rules apply (e.g., balancing, decomposition). Basic d/f b/n the two is that a physical DFD contains additional details that describe how system will be built.

Steps of transforming logical DFDs to physical DFDs

Steps	Explanation
1. Add implementation references.	Using the existing logical DFD, add the way in which the data stores, data flows, and processes will be implemented to each component.
2. Draw a human-machine boundary.	Draw a line to separate the automated parts of the system from the manual parts.
3. Add system-related data stores, data flows & processes.	Add system-related data stores, data flows, & processes to the model (components that have little to do with the business process).
4. Update the data elements in the data flows	Update the data flows to include system-related data elements.

Chapter 5- Structured Design

5.7. Algorithm/Program Design



Physical DFD for shop manager approval of custom drone order level 1 DFD.

Dennis, Wixon & Roth, 2019, p.321

Chapter 5- Structured Design

5.7. Algorithm/Program Design

Design Programs: Don't jump into coding, w/o much thought/planning. As this can lead to disastrous results: **inefficient programs, code that does not work with other code, and a system that doesn't do what it's supposed to do.**

- Instead, analysts should first take time in the design phase to create a maintainable system: analysts should create a design that is modular & flexible.
- To do this, analysts can design programs in a **top-down modular approach**, using a variety of program design techniques.

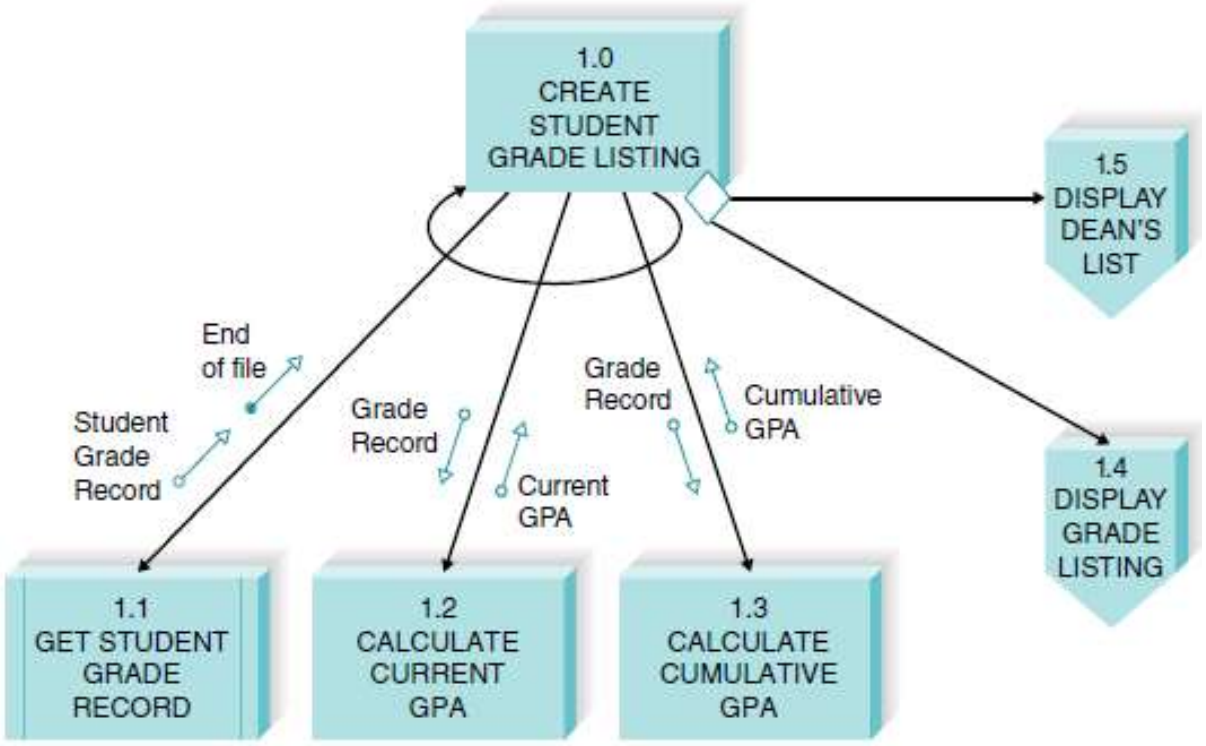
Structured Chart: **helps the analyst design the program for the new system**.

- It shows all the components of code that must be included in a program at a high level, arranged in a hierarchical format that implies **sequence** (in what order components are invoked), **selection** (under what condition a module is invoked), and **iteration** (how often a component is repeated). The components are usually read from top to bottom, left to right, and they are numbered by a hierarchical numbering scheme in which lower levels have an additional level of numbering (e.g., 3rd level of modules would be numbered 1.1.1, 1.1.2, 1.1.3...).
- helpful in the building of many types of systems because they emphasize structure and reusability, characteristics of any good application

Chapter 5- Structured Design

5.7. Algorithm/Program Design… Structured Chart

Example: Structured chart for an app that display student lists along with semester & cumulative GPA (Dennis, Wixon & Roth, 2019, p.329



Chapter 5- Structured Design

5.7. Algorithm/Program Design… Structured Chart Elements

Structured Chart Elements (Dennis, Wixon & Roth, 2019, p.328)

Structure Chart Element	Purpose	Symbol
Every *module*: • Has a number. • Has a name. • Is a control module if it calls other modules below it. • Is a subordinate module if it is controlled by a module at a higher level.	Denotes a logical piece of the program	1.2 CALCULATE CURRENT GPA
Every *library module* has: • A number. • A name. • Multiple instances within a diagram.	Denotes a logical piece of the program that is repeated within the structure chart	1.1 GET STUDENT GRADE RECORD
A *loop*: • Is drawn with a curved arrow. • Is placed around lines of one or more modules that are repeated.	Communicates that a module(s) is repeated	
A *conditional line*: • Is drawn with a diamond. • includes modules that are invoked on the basis of some condition.	Communicates that subordinate modules are invoked by the control module based on some condition	
A *data couple*: • Contains an arrow. • Contains an empty circle. • Names the type of data that are being passed. • Can be passed up or down. • Has a direction that is denoted by the arrow.	Communicates that data are being passed from one module to another	
A *control couple*: • Contains an arrow. • Contains a filled-in circle. • Names the message or flag that is being passed. • Should be passed up, not down. • Has a direction that is denoted by the arrow.	Communicates that a message or a system flag is being passed from one module to another	
An *off-page connector*: • Is denoted by the hexagon. • Has a title. • Is used when the diagram is too large to fit everything on the same page.	Identifies when parts of the diagram are continued on another page of the structure chart	PRINT GRADE LISTING
An *on-page connector*: • Is denoted by the circle. • Has a title. • Is used when the diagram is too large to fit everything in the same spot on a page.	Identifies when parts of the diagram are continued somewhere else on the same page of the structure chart	PRINT GRADE LISTING

Chapter 5- Structured Design

5.7. Algorithm/Program Design… Building the Structure Chart

Step 1: Identify Modules and Levels:

- 1st, identify DFD processes & convert them into structure chart modules.
- Modules should perform only one function; if a process contains more than one function, it should be split into more than one module.
- Identify reusable processes from DFDs; if so, represent them as library modules

Step 2: Identify Special Connections:

- 2nd, add loops & conditional lines to represent modules that are repeated/optional.

Step 3: Add Couples:

- Identify info that has to pass among modules like data attributes (denoted by an empty circled arrow) or special control parameters (denoted by filled in circle arrow).
- The arrowheads on arrows indicate which way info is passed along. DFD data flows provide some guidance about couples to add, b/c info that flows in & out of DFD processes likely will also flow in & out of the corresponding structure chart modules.

Revise/fine-tune/ the Structure Chart/

1st , look at process descriptions [DD] to check if there processes not considered.

2nd, examine data model to confirm that right records & specific fields have been passed using the data couples

Chapter 5- Structured Design

5.7. Algorithm/Program Design… Building the Structure Chart

Design Guidelines

- ❑ High-quality structure charts result in programs that are modular, reusable, and easy to implement.
- ❑ Measures of good design include cohesion, coupling, & appropriate levels of fan-in &/or fan-out.

Build Modules With High Cohesion

Build Loosely Coupled Modules

Create High Fan-In: Fan-in describes # of control modules that communicate with a subordinate; a module with high fan-in has many d/t control modules that call it. This is a very good situation b/c high fan-in indicates that a module is reused in many places on structure chart, suggesting the module contains well-written generic code.

Avoid High Fan-Out: avoid a large # of subordinates associated with a single control.

Assess the Chart for Quality

Checklist for structure chart quality

(Dennis, Wixon & Roth, 2019, p.342

- ✓ Library modules have been created whenever possible.
- ✓ The diagram has a high fan-in structure.
- ✓ Control modules have no more than seven subordinates.
- ✓ Each module performs only one function (high cohesion).
- ✓ Modules sparingly share information (loose coupling).
- ✓ Data couples that are passed are actually used by the accepting module.
- ✓ Control couples are passed from "low to high."
- ✓ Each module has a reasonable amount of code associated with it.

Chapter 5- Structured Design

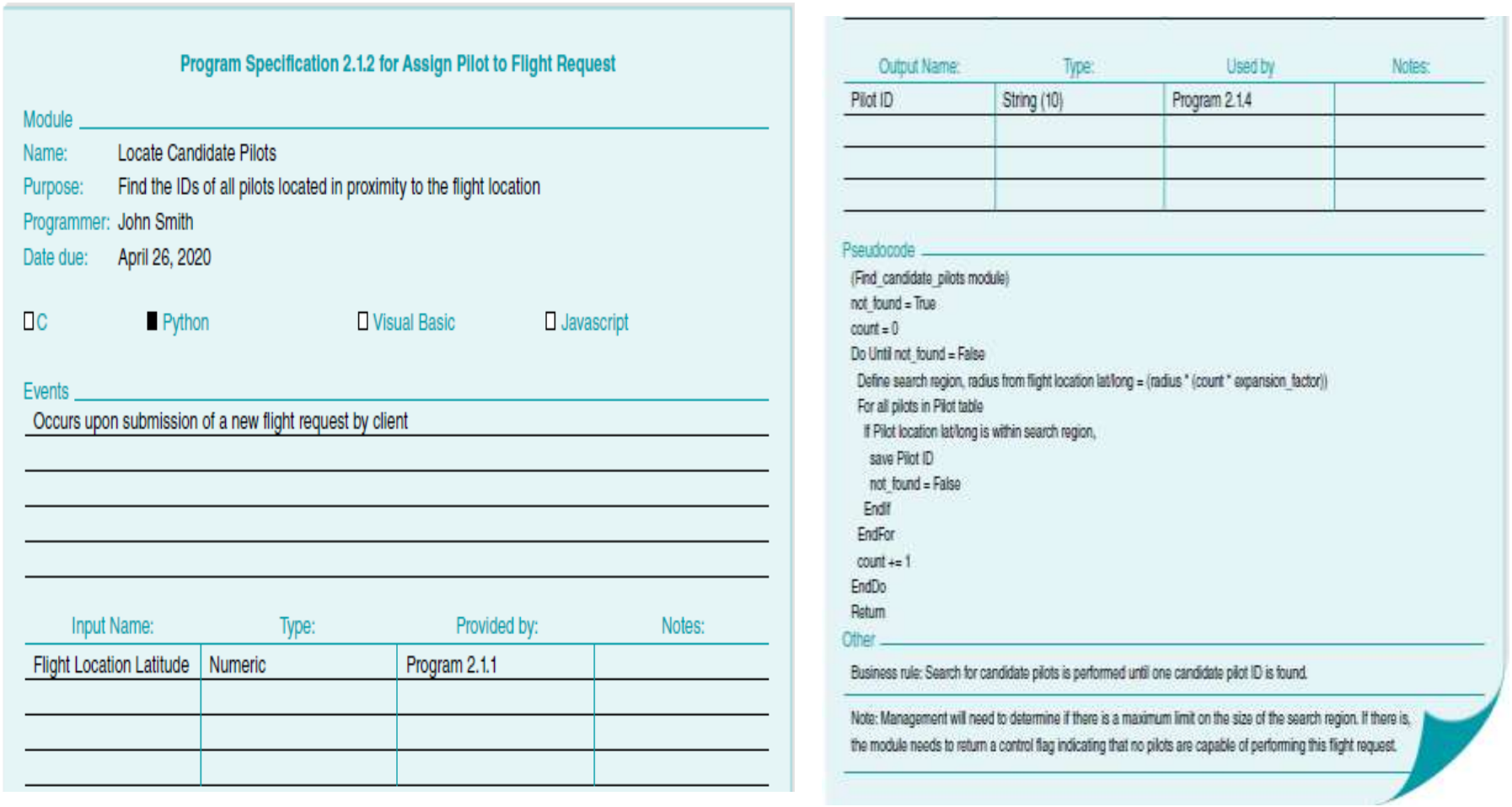
5.7. Algorithm/Program Design…

Program Specification (PS)

- Struc. chart: provides the big picture of how program should be put together
- PS written document including module descriptions & instructions on how to program pieces of code
- Typically, project team members write one program spec for each module on structure chart & then pass them along to programmers, who write the code during the implementation phase of the project.
- Specifications must be very clear & easy to understand, or else programmers will be slowed down trying to decipher vague or incomplete instructions
- Program specifications can pinpoint design problems that exist in the structure chart. At a high level, the structure chart may make sense, but when the analyst actually begins writing the detail behind the modules, he or she may find better ways for arranging the modules or may uncover missing or unnecessary couples
- There is no formal syntax for a program specification, so every organization uses its own format (see example on next slide)

Chapter 5- Structured Design

5.7. Algorithm/Program Design… Program Specification …



Program Specification 2.1.2 for Assign Pilot to Flight Request

Module

Name: Locate Candidate Pilots

Purpose: Find the IDs of all pilots located in proximity to the flight location

Programmer: John Smith

Date due: April 26, 2020

☐ C ■ Python ☐ Visual Basic ☐ Javascript

Events

Occurs upon submission of a new flight request by client

Input Name:	Type:	Provided by:	Notes:
Flight Location Latitude	Numeric	Program 2.1.1	

Output Name:	Type:	Used by	Notes:
Pilot ID	String (10)	Program 2.1.4	

Pseudocode

```
(Find_candidate_pilots module)
not_found = True
count = 0
Do Until not_found = False
  Define search region, radius from flight location lat/long = (radius * (count * expansion_factor))
  For all pilots in Pilot table
    If Pilot location lat/long is within search region,
      save Pilot ID
      not_found = False
    EndIf
  EndFor
  count += 1
EndDo
Return
```

Other

Business rule: Search for candidate pilots is performed until one candidate pilot ID is found.

Note: Management will need to determine if there is a maximum limit on the size of the search region. If there is, the module needs to return a control flag indicating that no pilots are capable of performing this flight request.

Program Specification form (Dennis, Wixon & Roth, 2019, p.347)